

CSE 4303

Data Structures

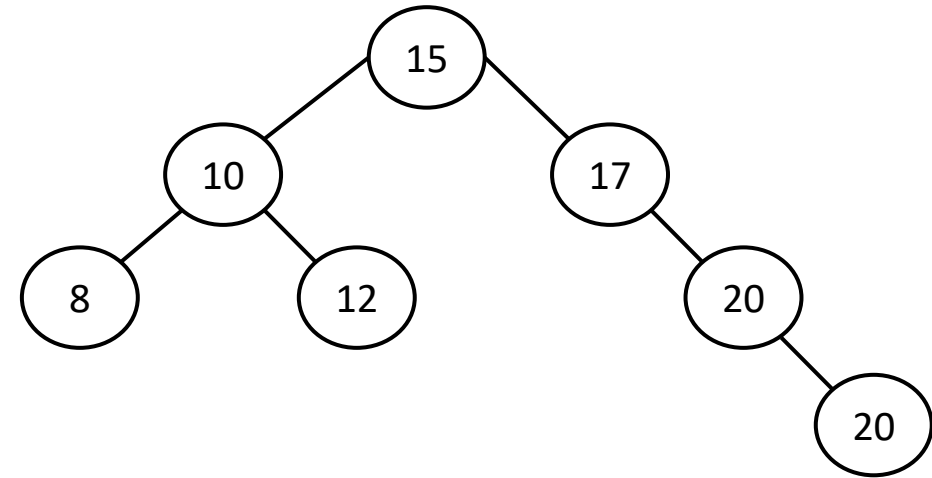
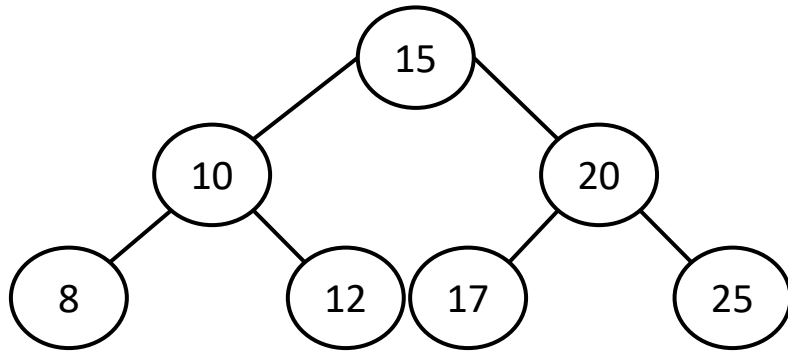
Topic: AVL Trees

Sabbir Ahmed

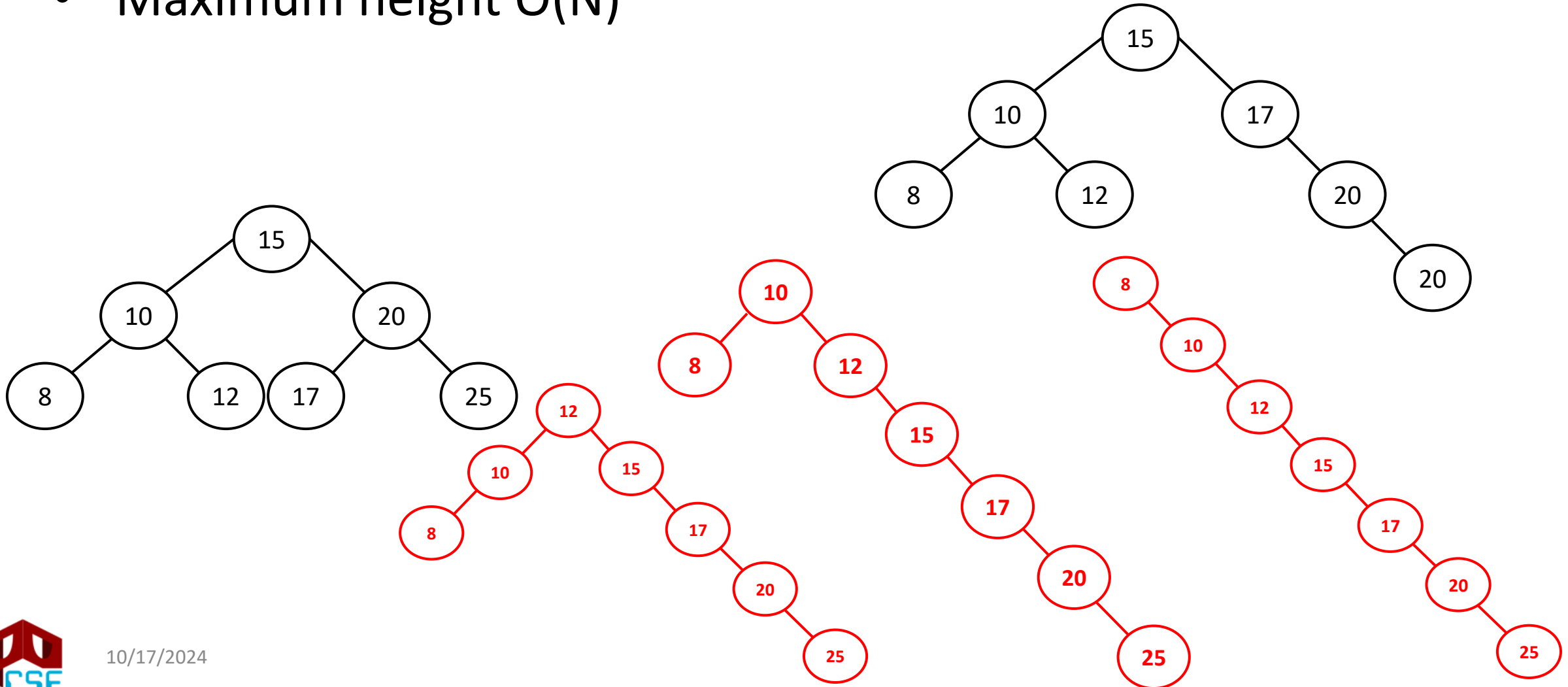
Assistant Prof | CSE | IUT

sabbirahmed@iut-dhaka.edu

- BST operations requires $O(\text{height})$ time.
- Minimum height $O(\log N)$



- BST operations requires $O(\text{height})$ time.
- Minimum height $O(\log N)$
- Maximum height $O(N)$

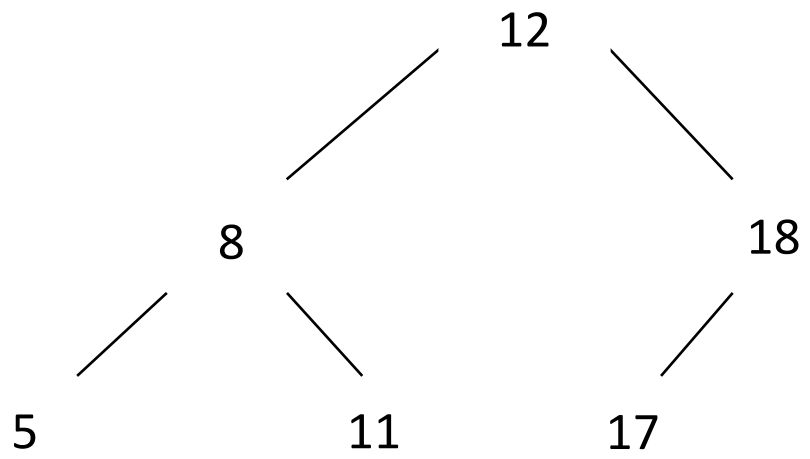


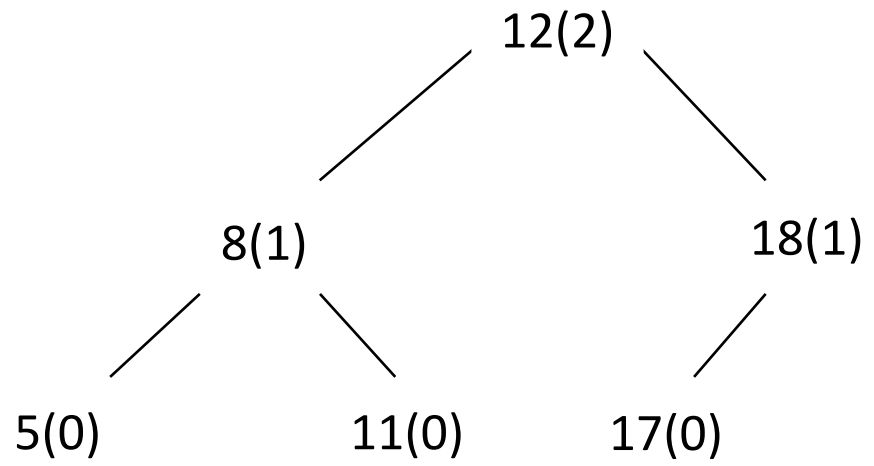
- Self-balancing BST can ensure height = $O(\log N)$
- Example: AVL Tree, Red-Black Tree, Splay Tree, etc.

- Self-balancing BST can ensure height = $O(\log N)$
- Example: AVL Tree, Red-Black Tree, Splay Tree, etc.
- Today's agenda: AVL Tree!
- Ensures every node is **Balanced**. But how?

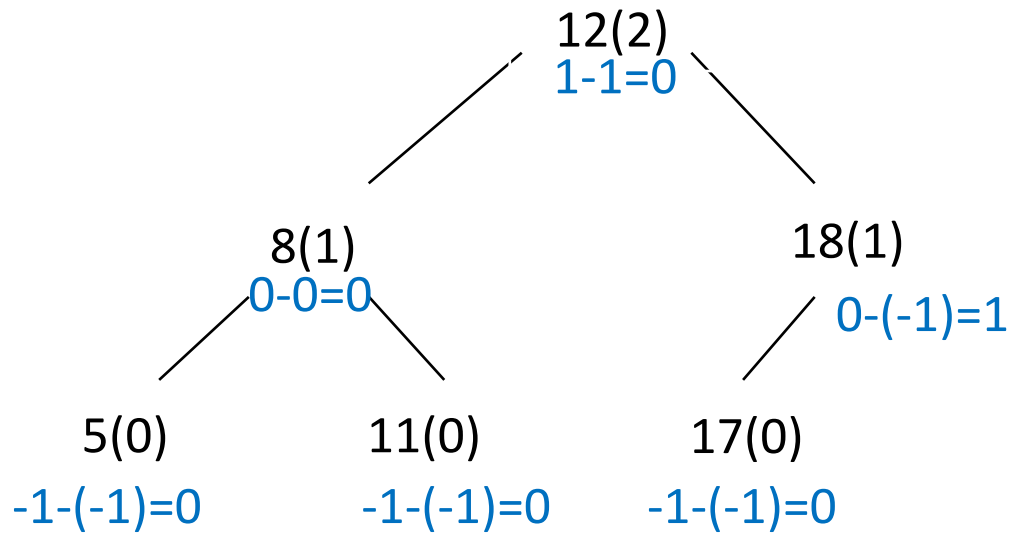
- Self-balancing BST can ensure height = $O(\log N)$
- Example: AVL Tree, Red-Black Tree, Splay Tree, etc.
- Today's agenda: AVL Tree!
- Ensures every node is **Balanced**. But how?
- Balance factor = $\text{height}(\text{left_subtree}) - \text{height}(\text{right_subtree})$
- If balance factor = $\{-1, 0, 1\}$ tree is balanced

- Self-balancing BST can ensure height = $O(\log N)$
- Example: AVL Tree, Red-Black Tree, Splay Tree, etc.
- Today's agenda: AVL Tree!
- Ensures every node is **Balanced**. But how?
- Balance factor = $\text{height}(\text{left_subtree}) - \text{height}(\text{right_subtree})$
- If balance factor = $\{-1, 0, 1\}$ tree is balanced
- What if the tree is 'imbalanced'?
- AVL tree fixes it using 'Rotation'





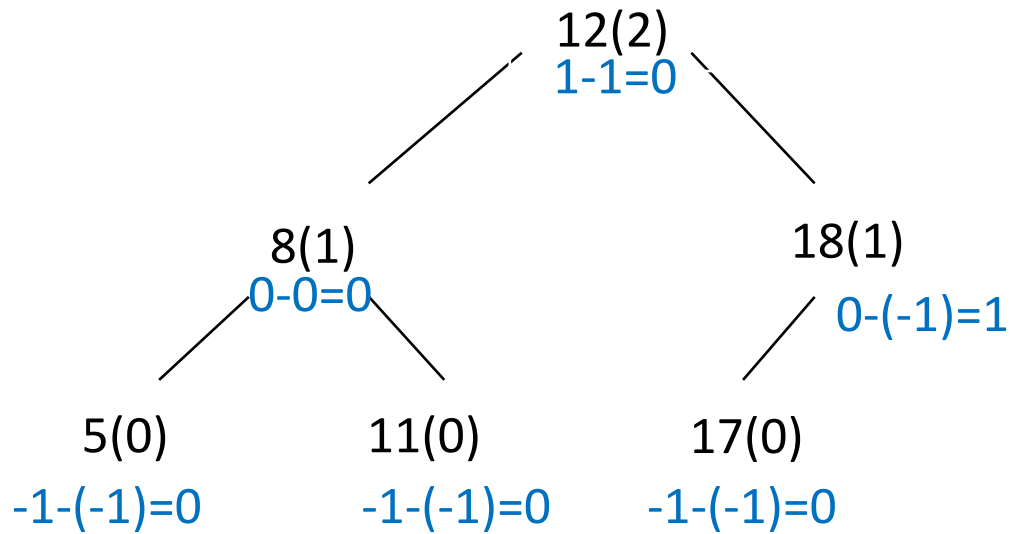
Node: data(height)



Node: data(height)

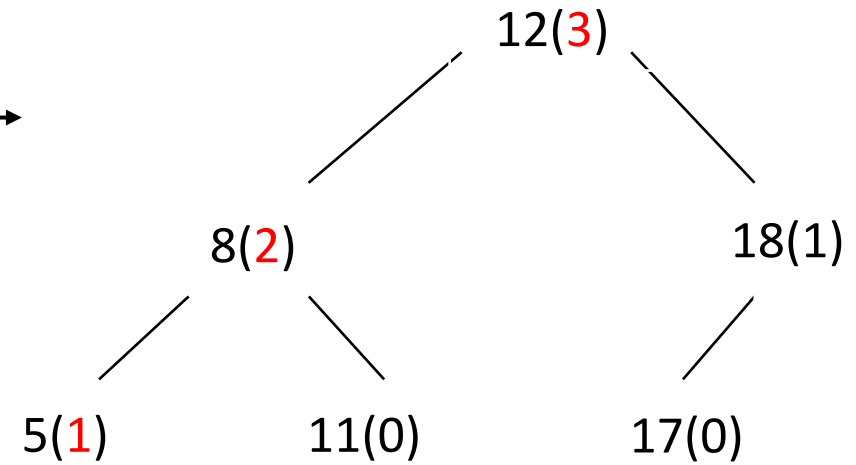
Balance factor= l_height- r_height

All nodes are balanced!

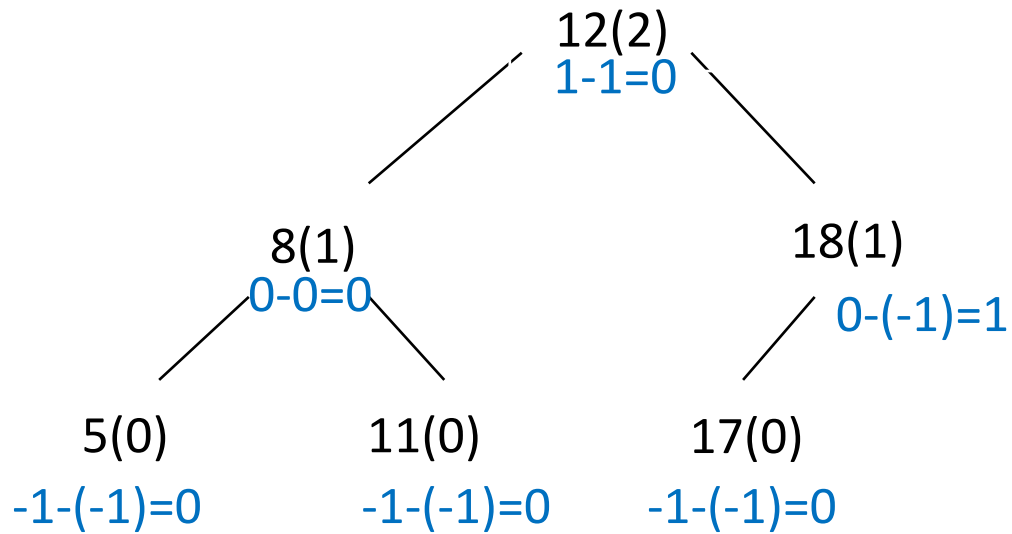


insert(4)

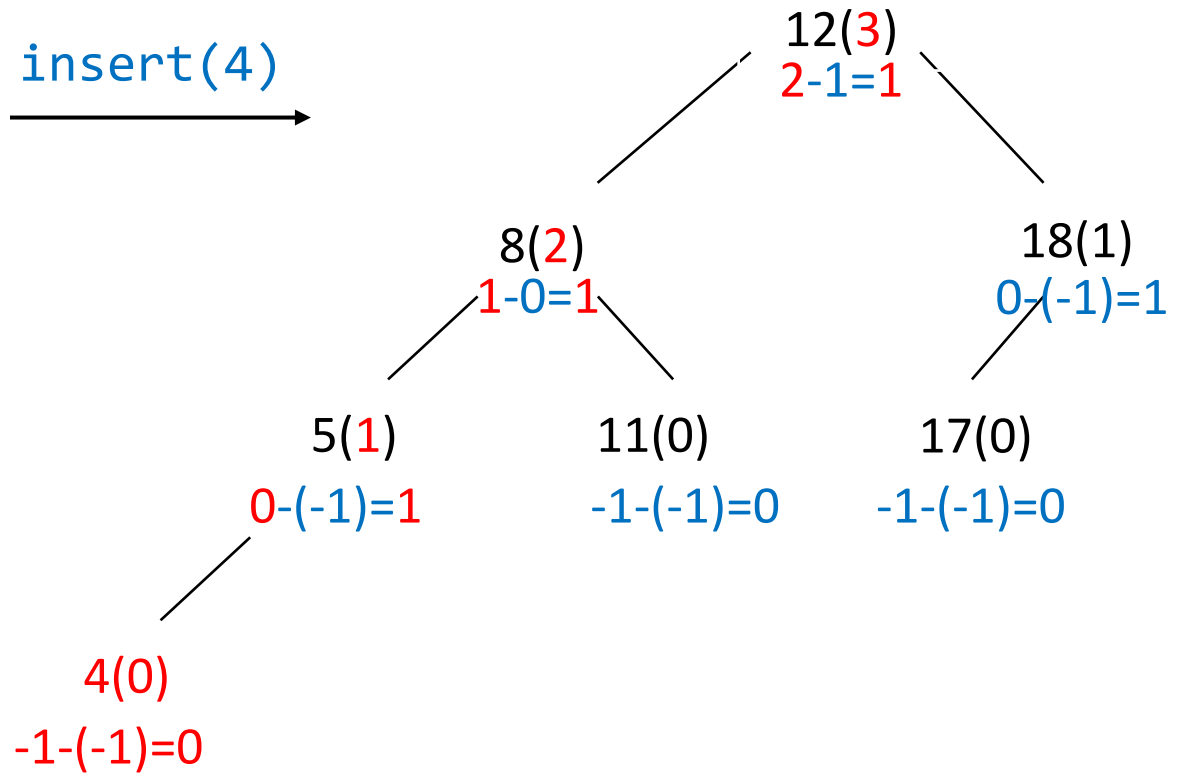
4(0)



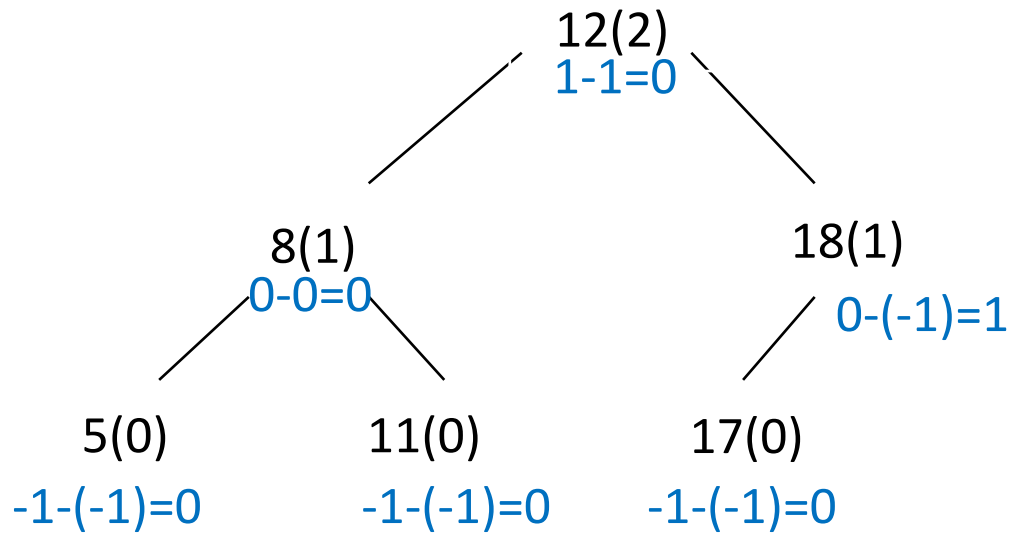
- After inserting newNode, update the heights of all ancestors upto root
 - For each node, $\max(l_h, r_h)+1$



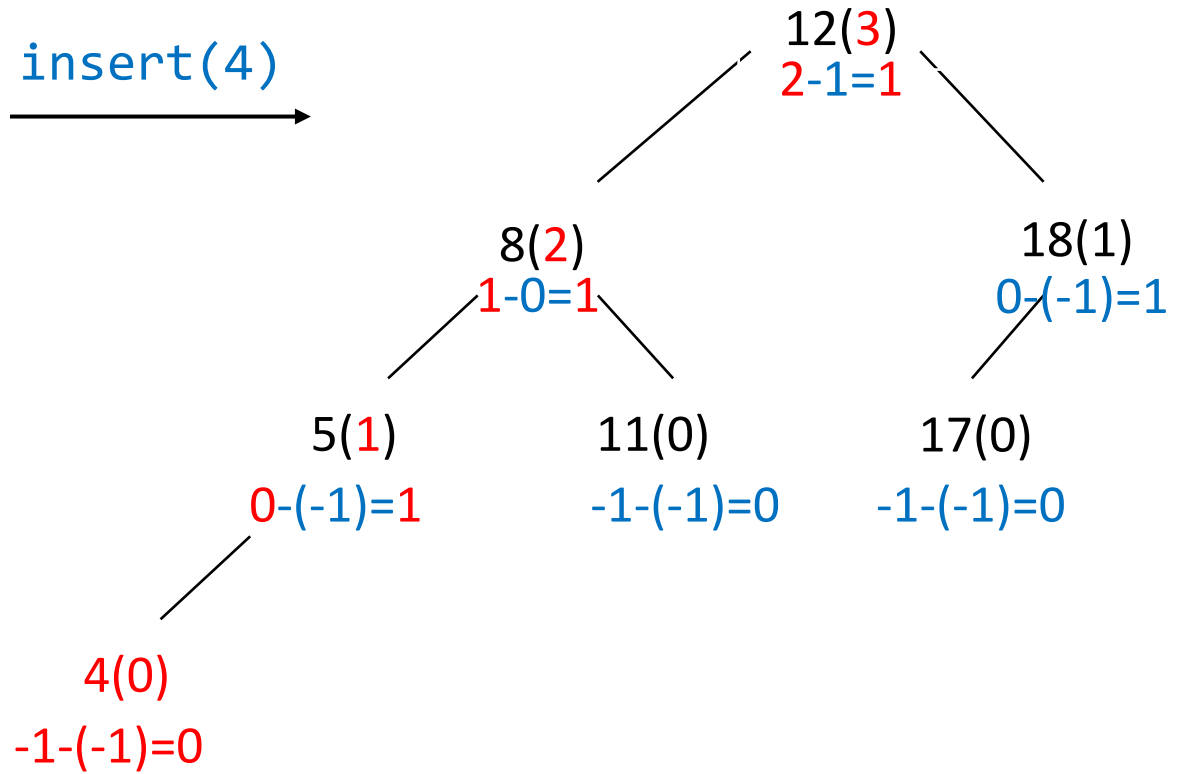
insert(4)



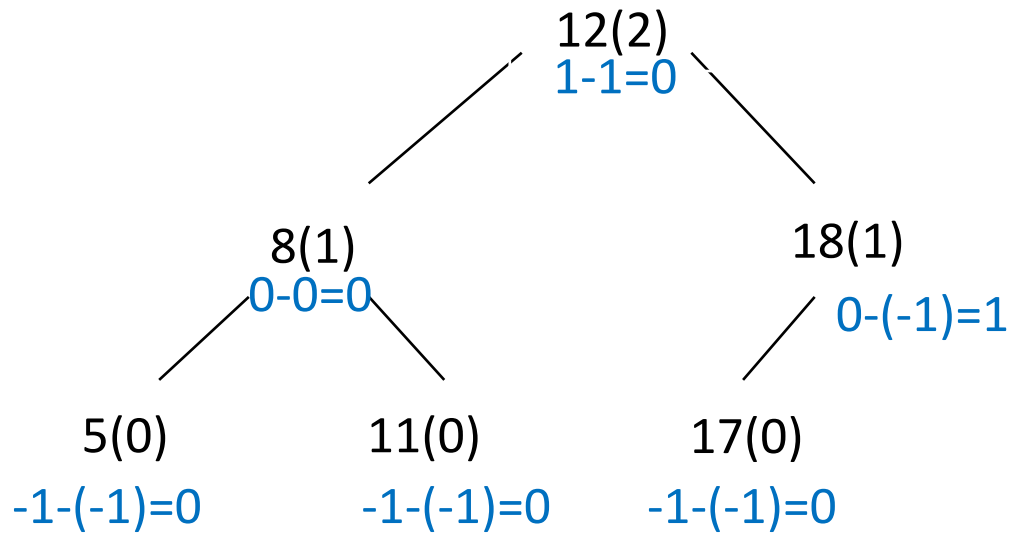
- Then check the balance factor for each node.
- If we find a violation, rotate that node.



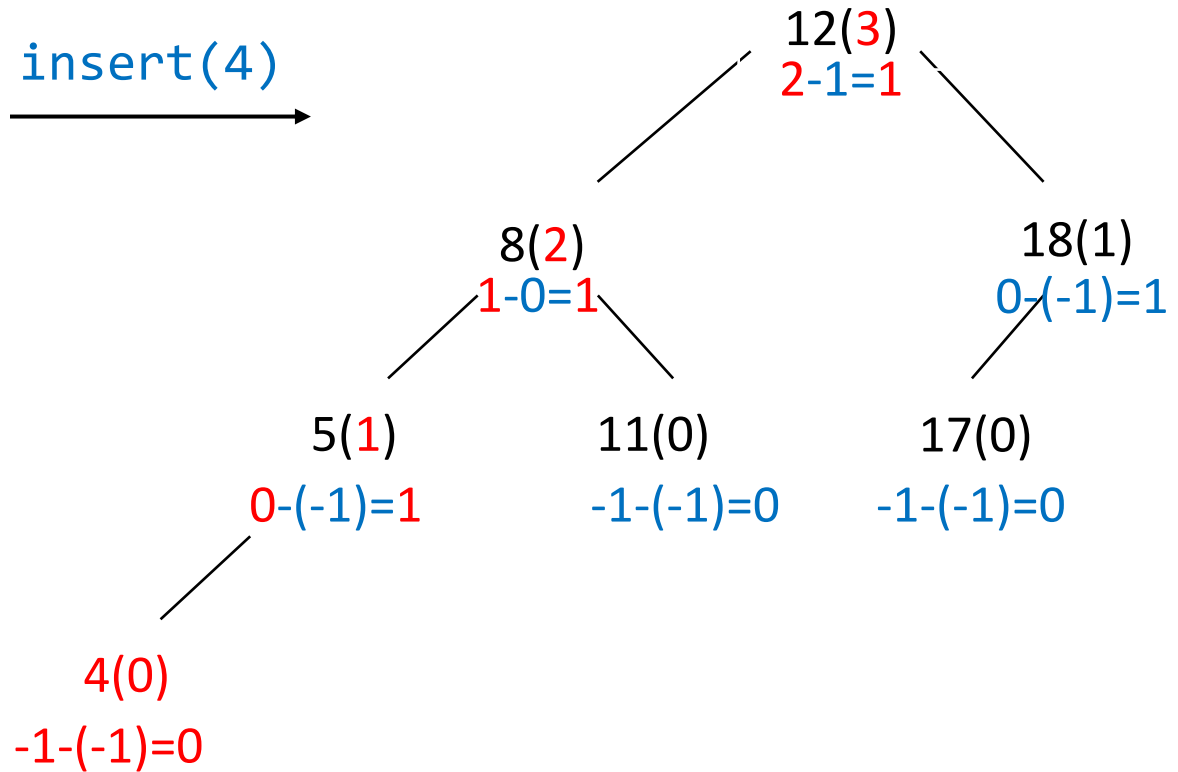
insert(4)



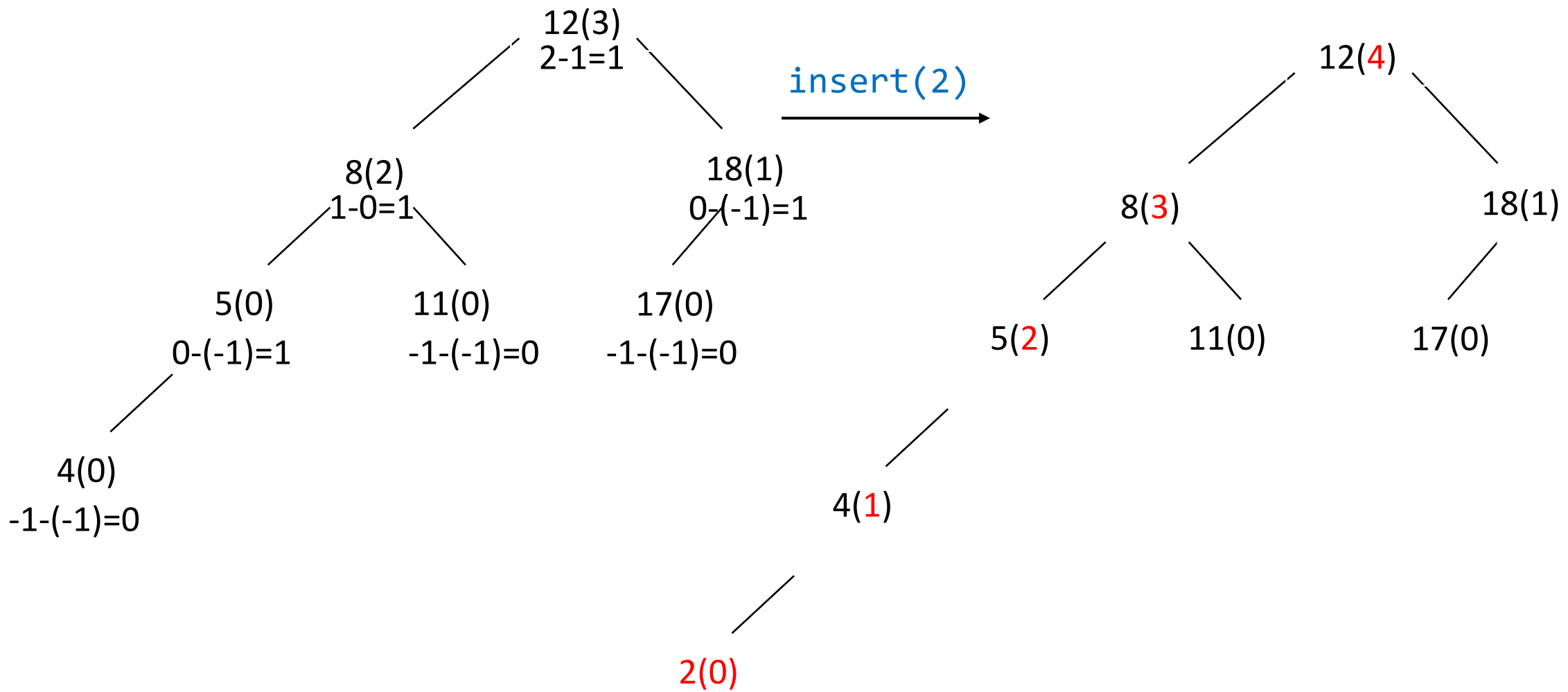
Balanced: balance factor = 0
 Left heavy: balance factor = 1
 Right heavy: balance factor = -1

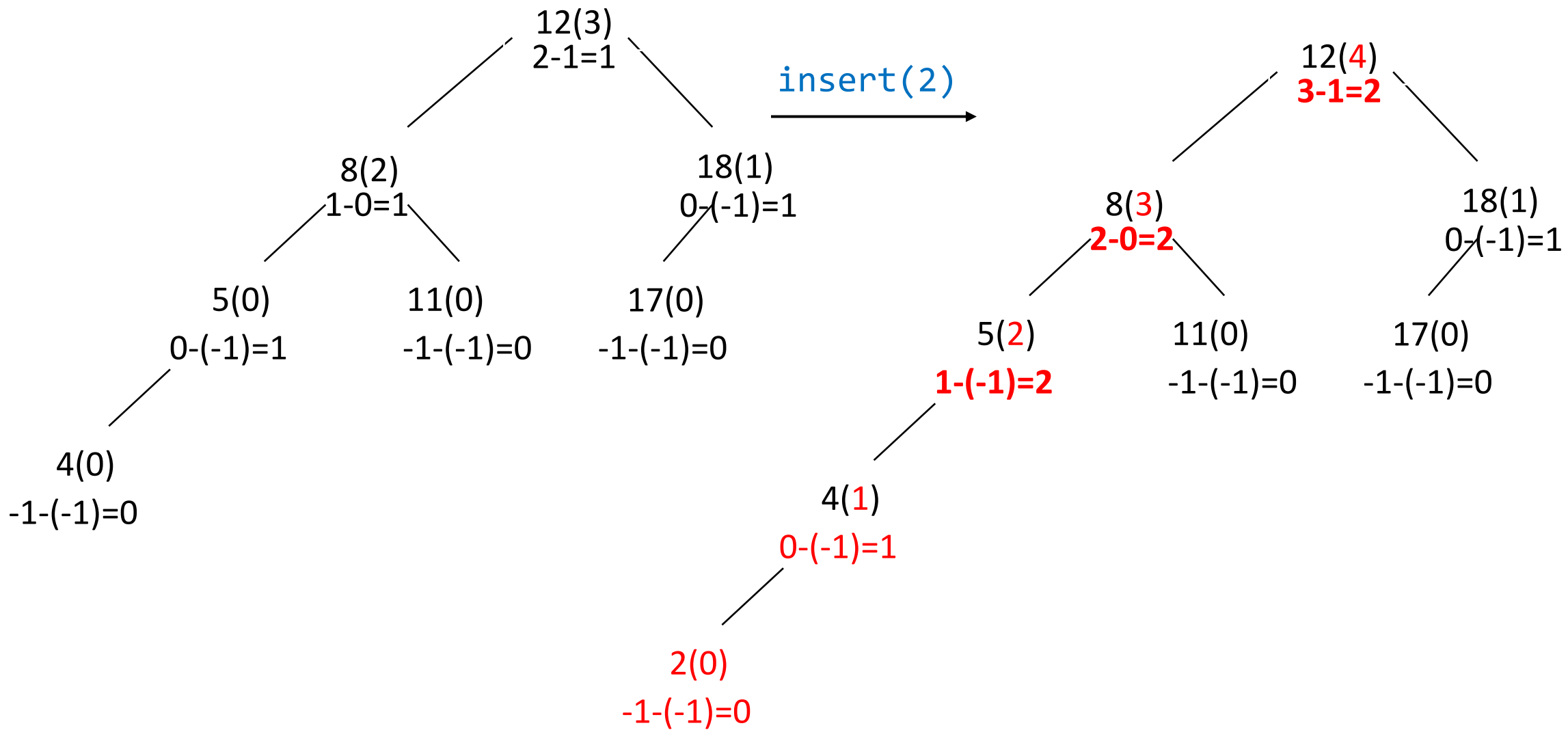


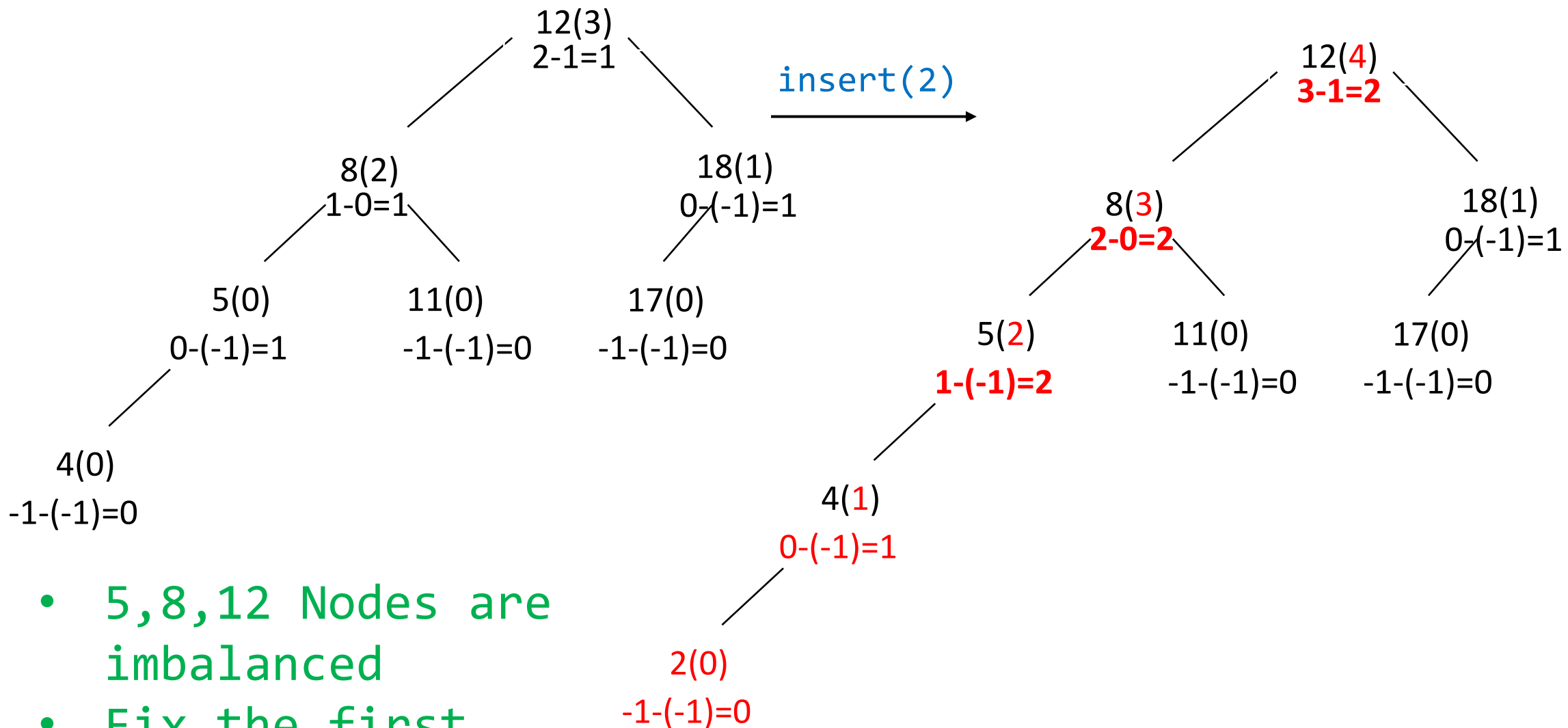
insert(4)



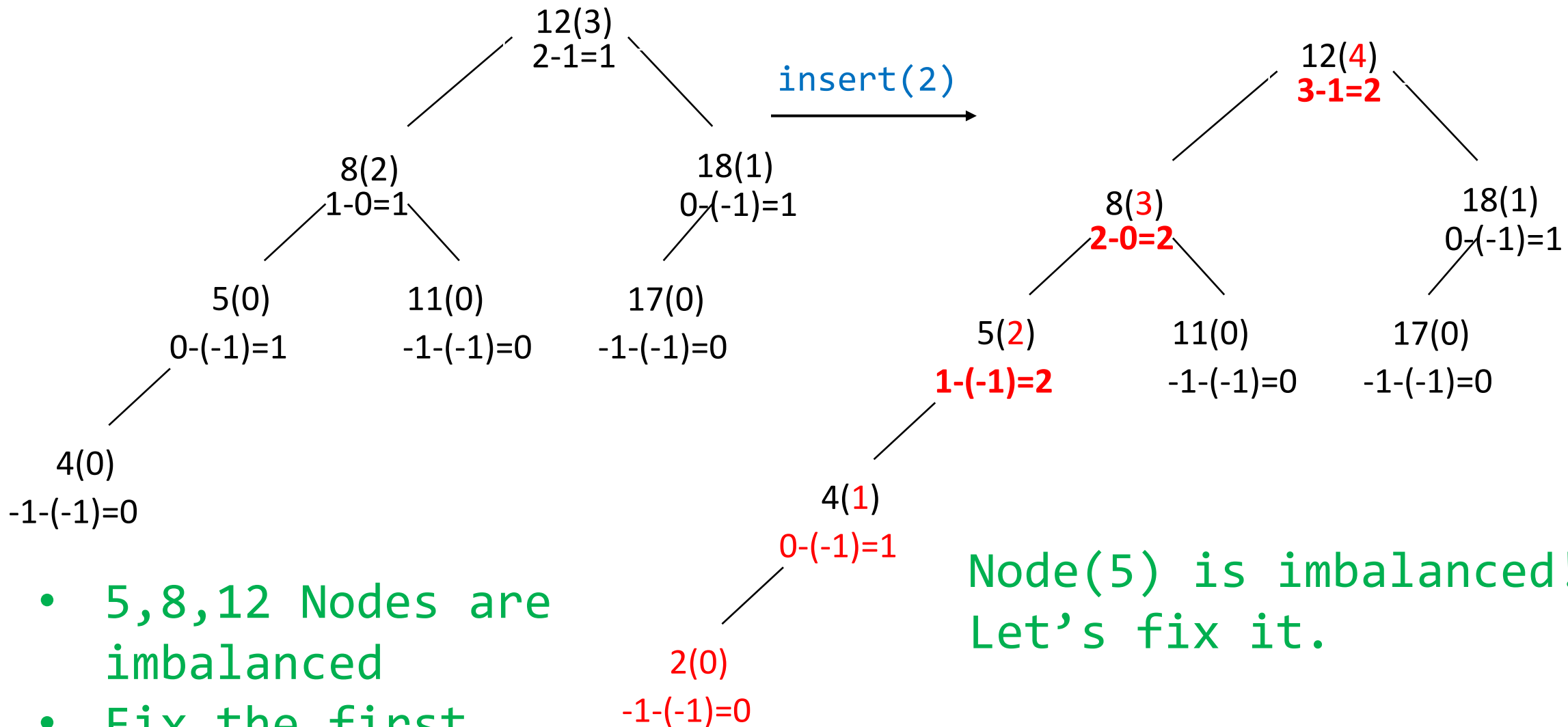
Balanced: balance factor = 0
 Left heavy: balance factor = 1
 Right heavy: balance factor = -1
 Imbalanced: >1 or <-1







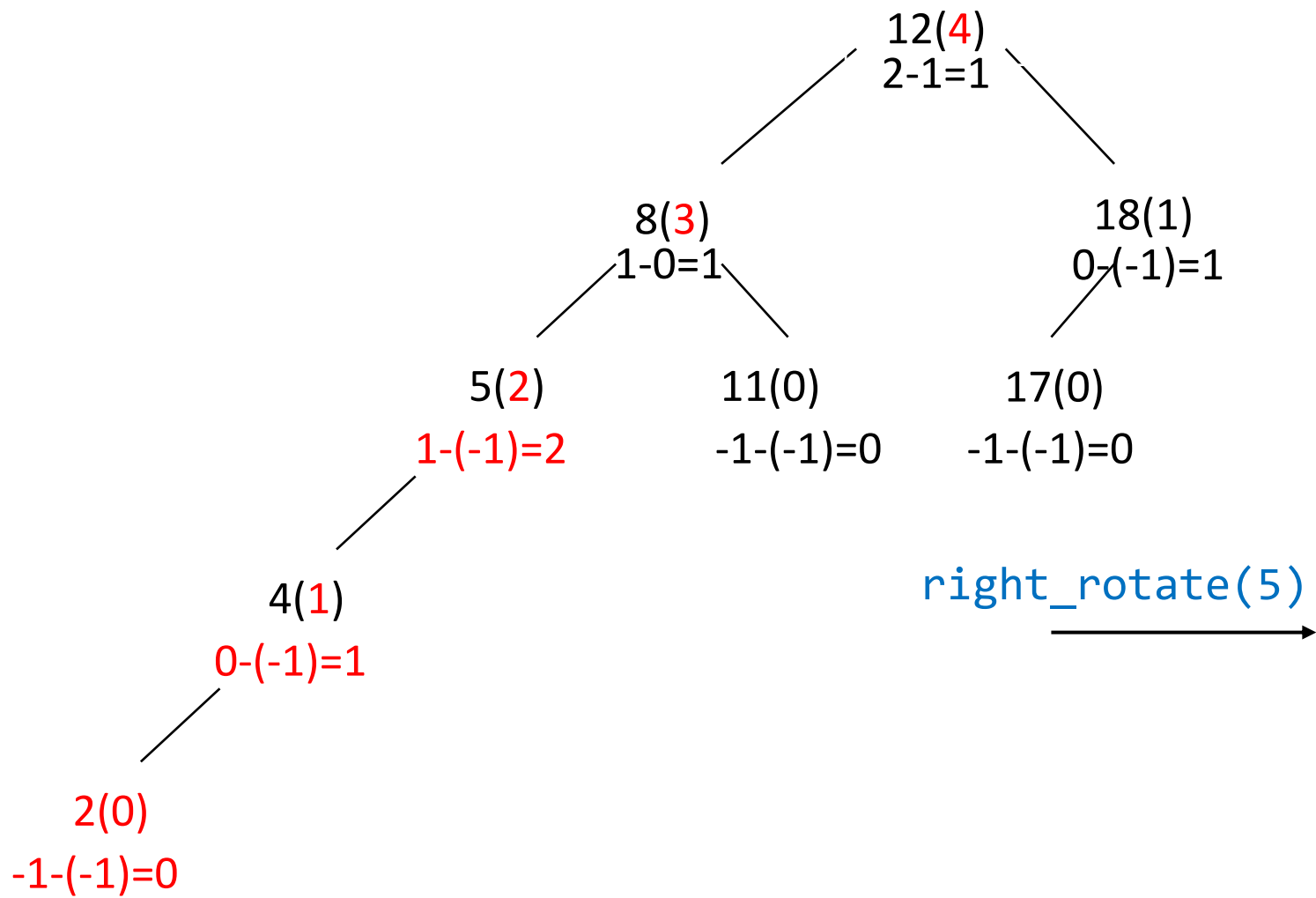
- 5, 8, 12 Nodes are imbalanced
- Fix the first one(5)

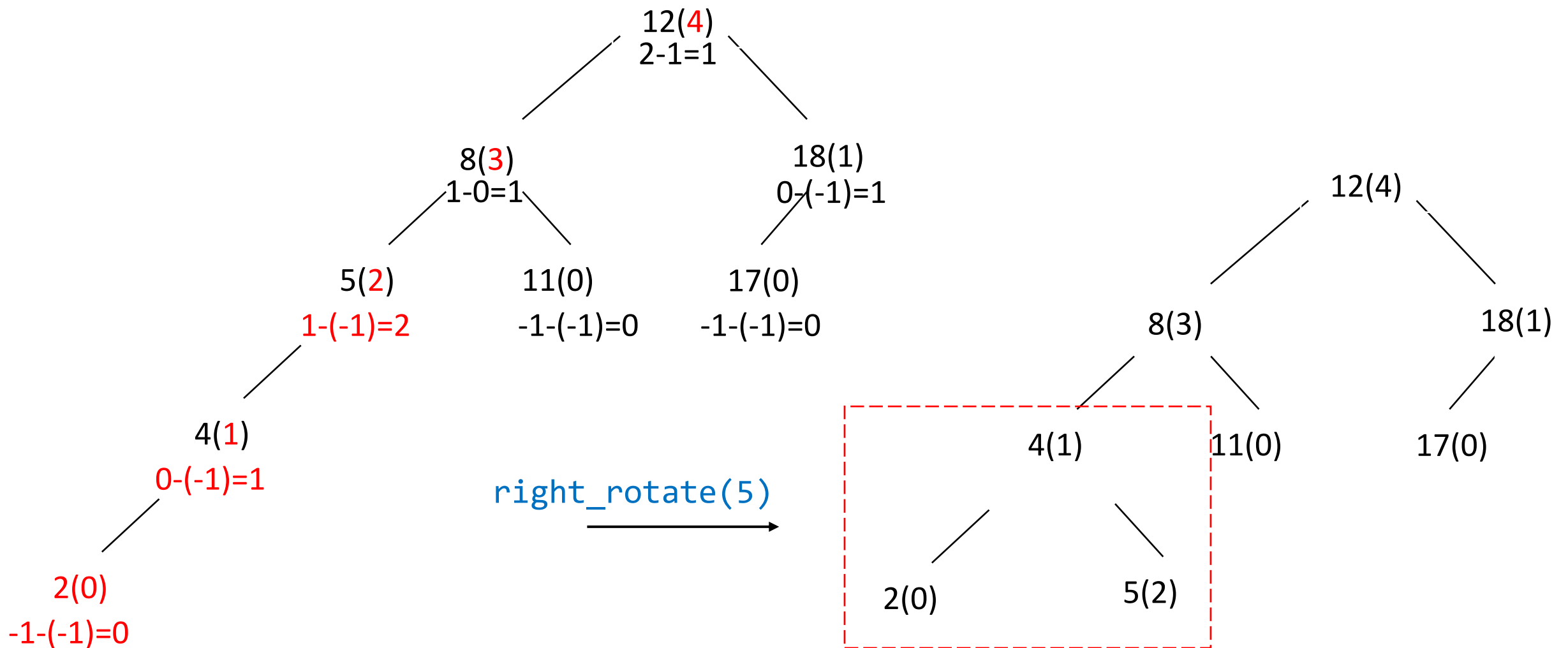


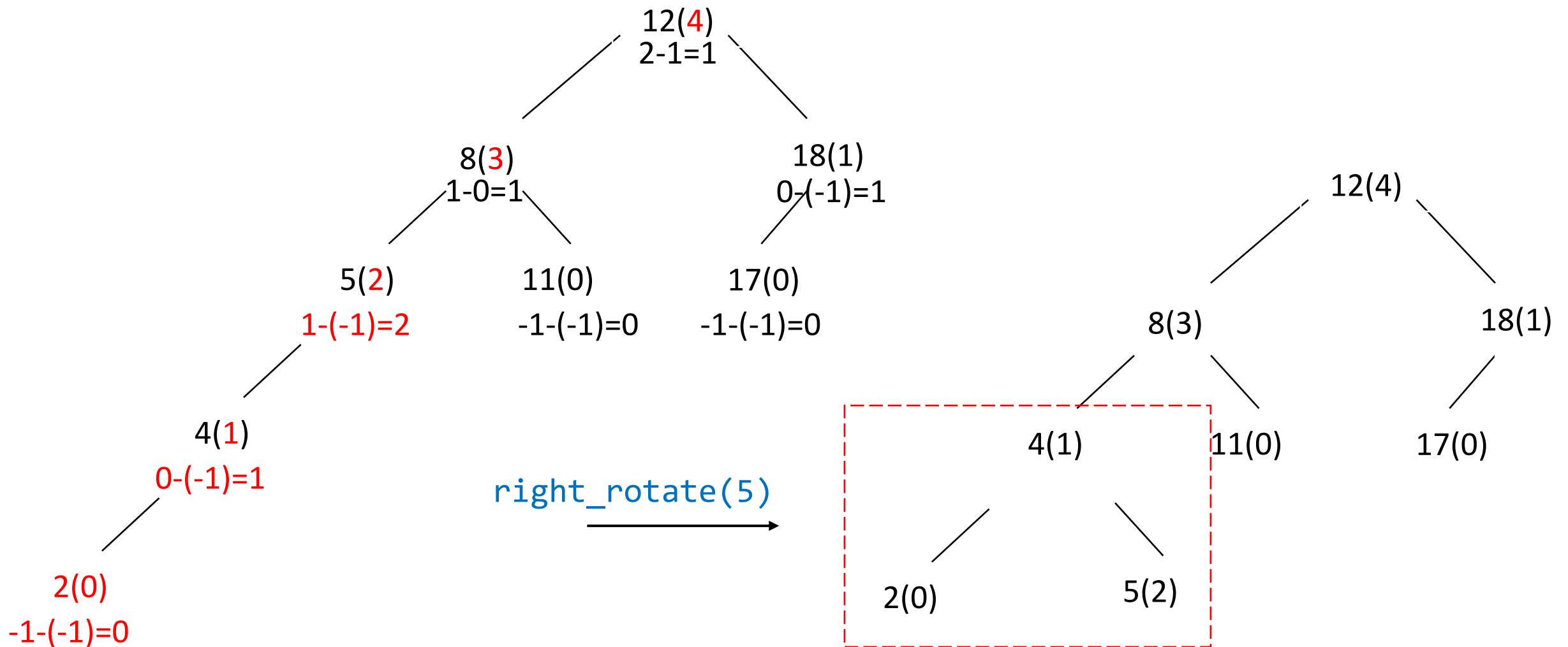
- 5,8,12 Nodes are imbalanced
- Fix the first one(5)

Node(5) is imbalanced!
Let's fix it.

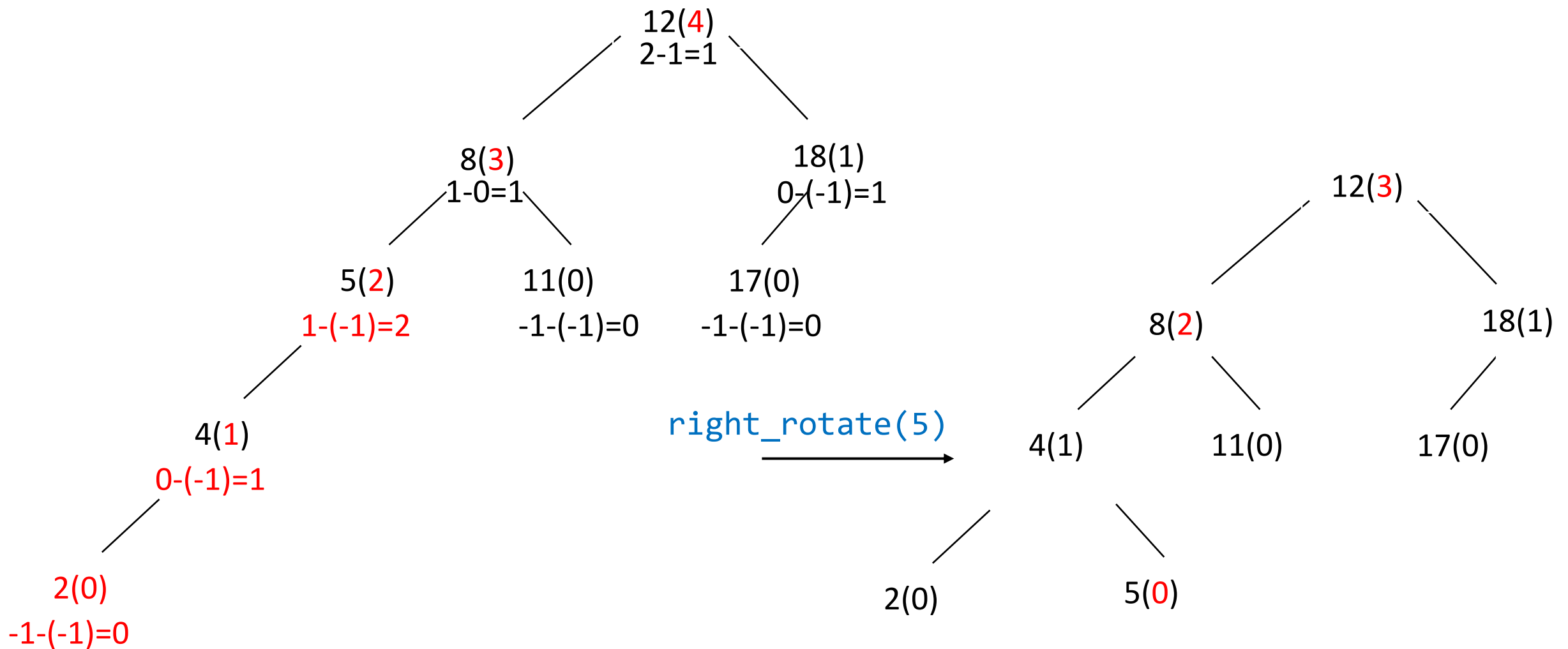
Once this is fixed,
entire tree is balanced.



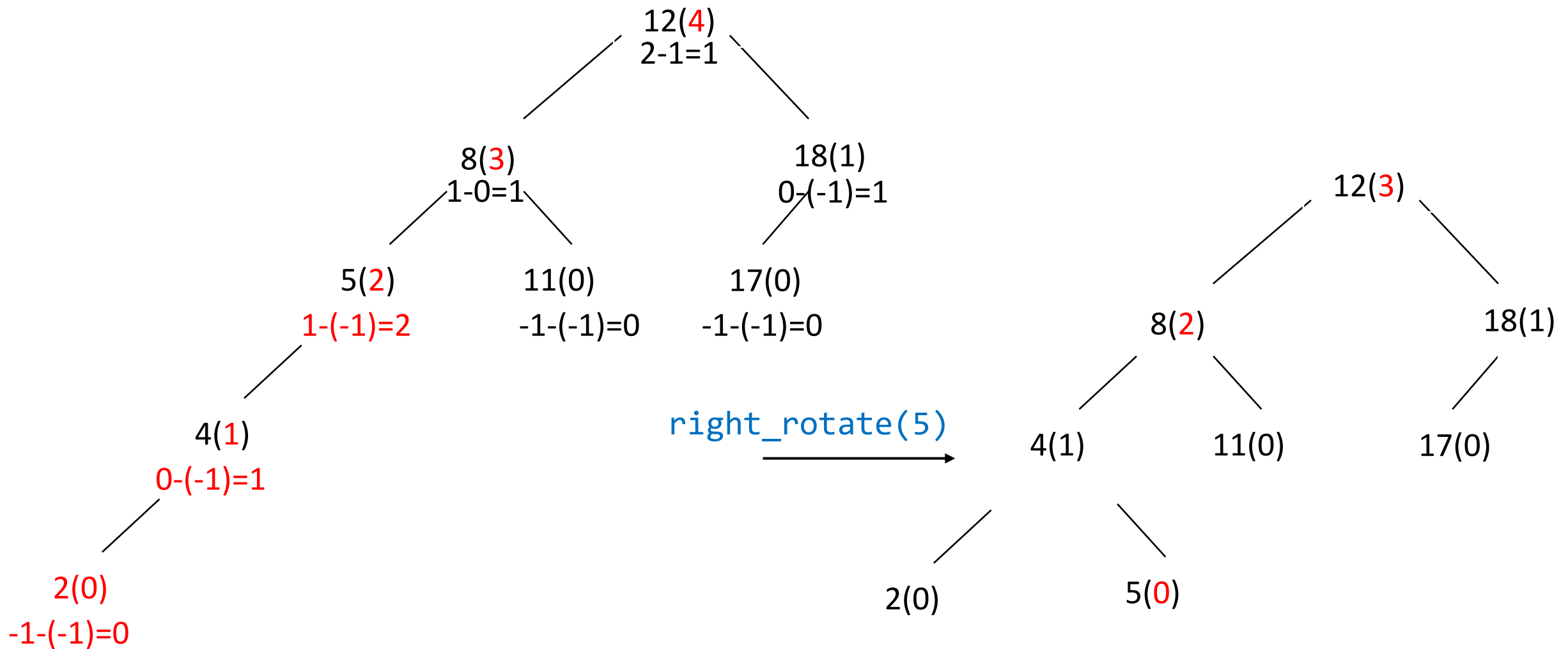




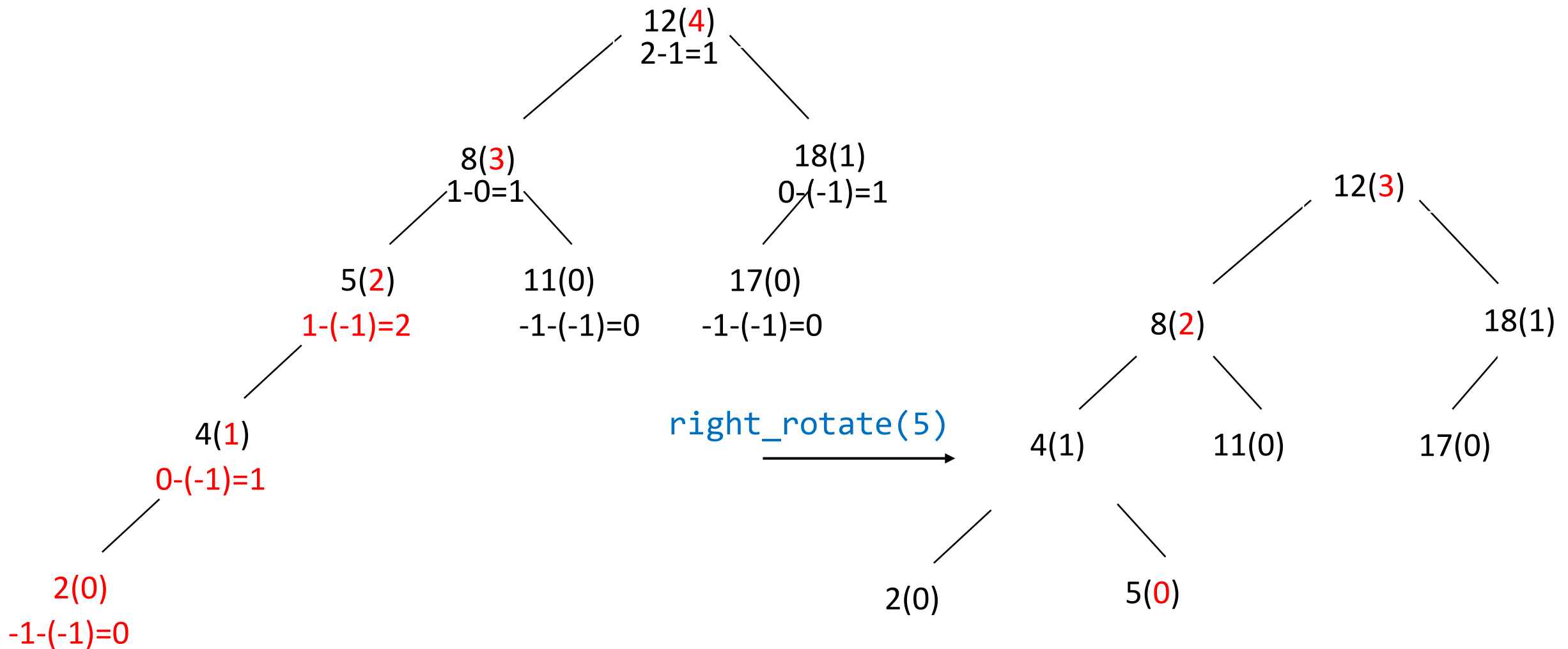
Need to update the heights again!



All ancestors of 5 are updated (if needed)

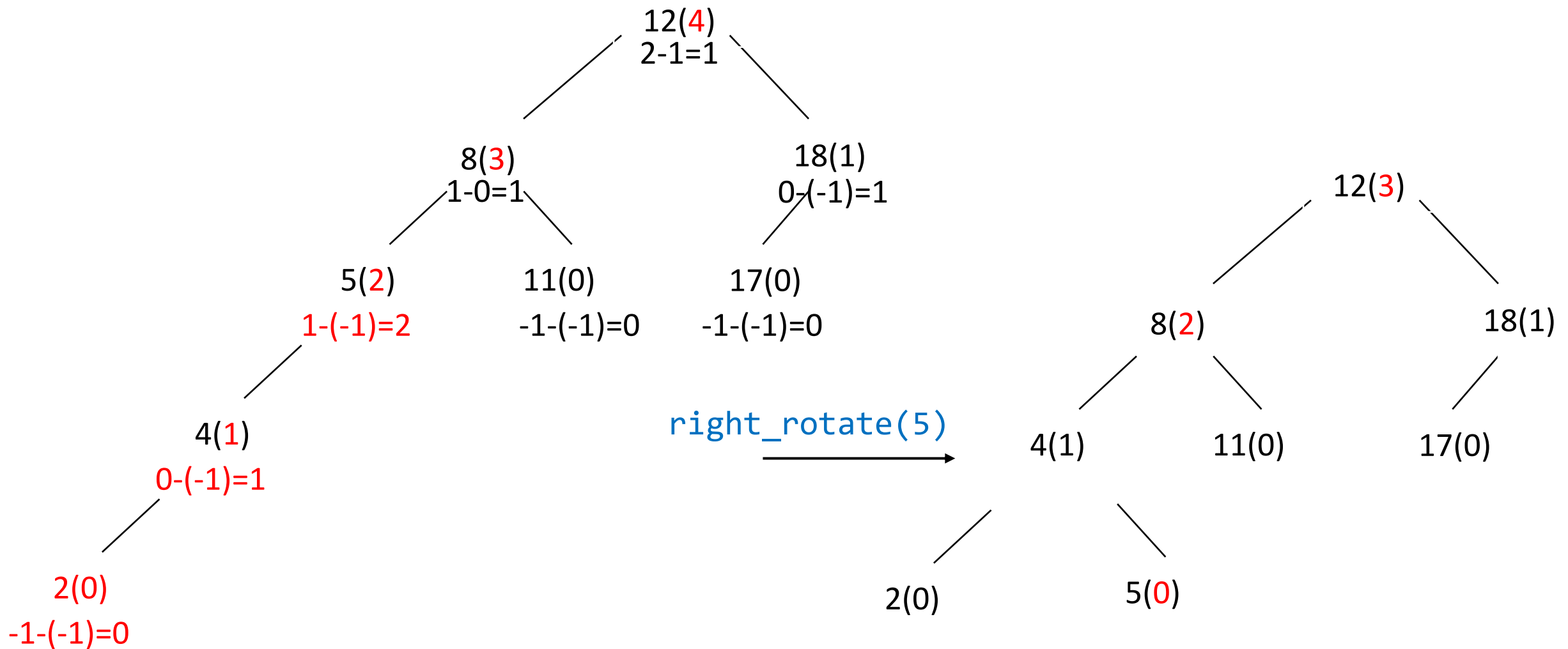


Once the first imbalanced node is balanced,
the tree is balanced.



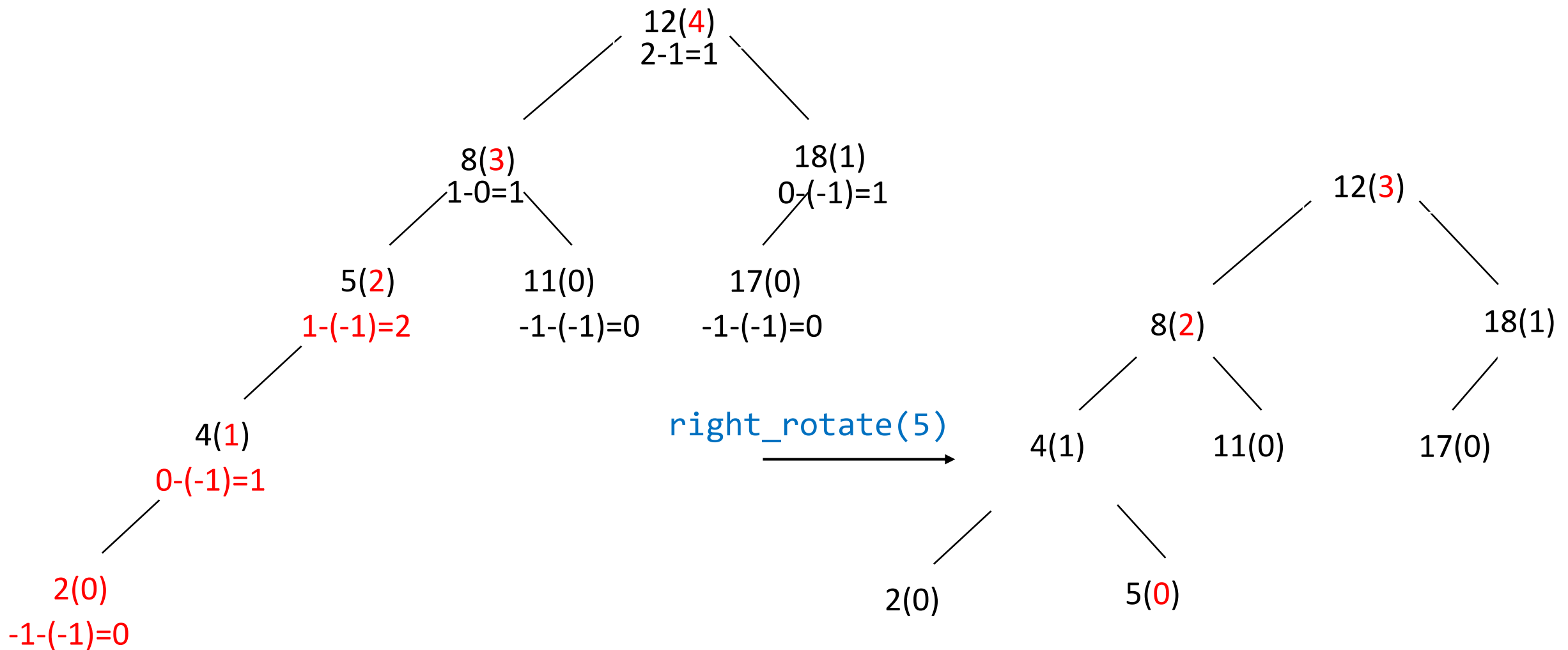
If we recalculate the Balance factor, we'll see no more violations!

Thus the $O(\log N)$ height is ensured.



Note: height is $O(\log N)$; not exactly $\log N$

Because, we don't ensure level-by-level, left-to-right insertion like complete binary trees!



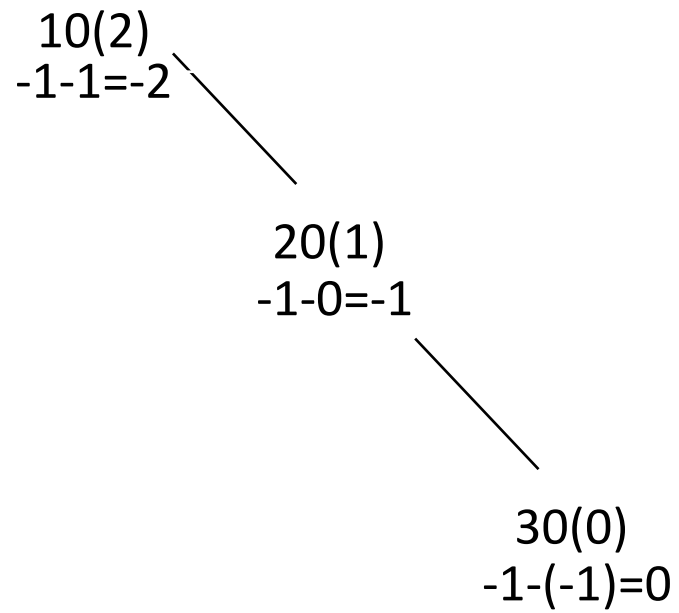
Note: height is $O(\log N)$; not exactly $\log N$

AVL tree ensures Tree height doesn't exceed $\sim 1.44 \log N$

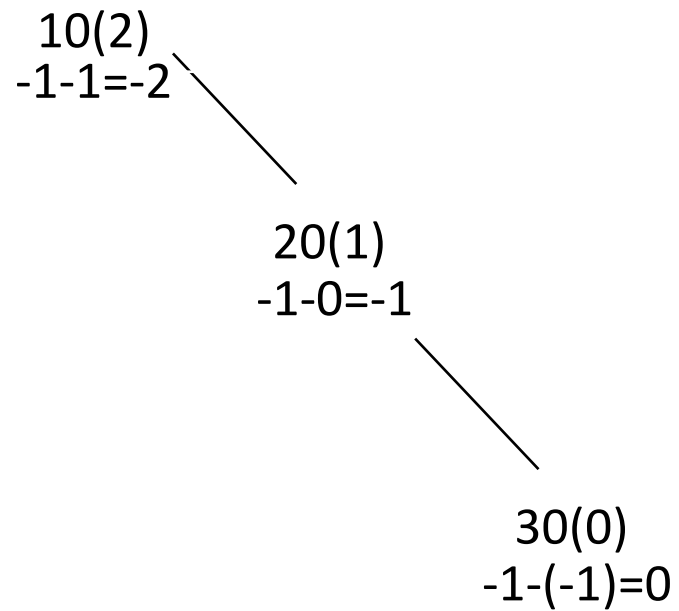
$$10(2)$$
$$-1-1=-2$$

$$20(1)$$
$$-1-0=-1$$

$$30(0)$$
$$-1-(-1)=0$$

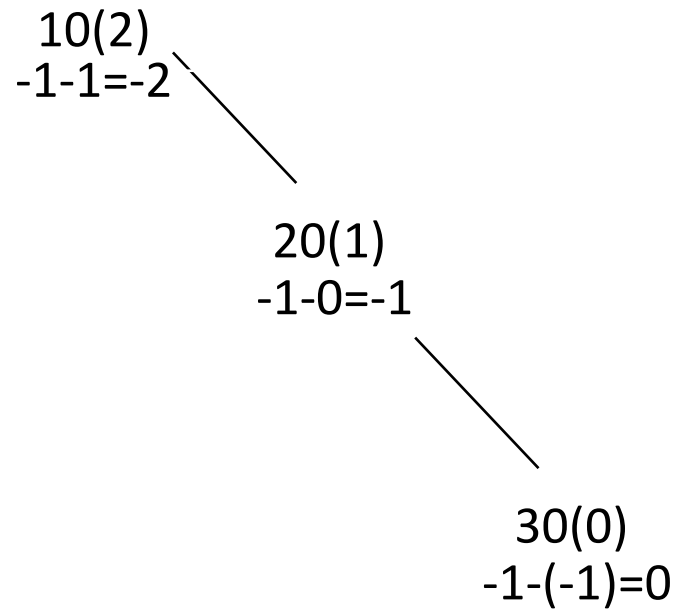


10 is imbalanced (z)
Z is Right-heavy



10 is imbalanced (z)
Z is Right-heavy

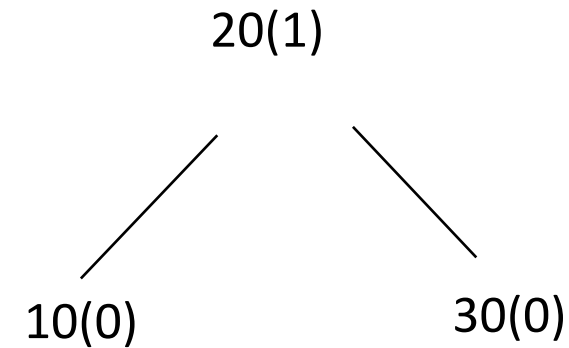
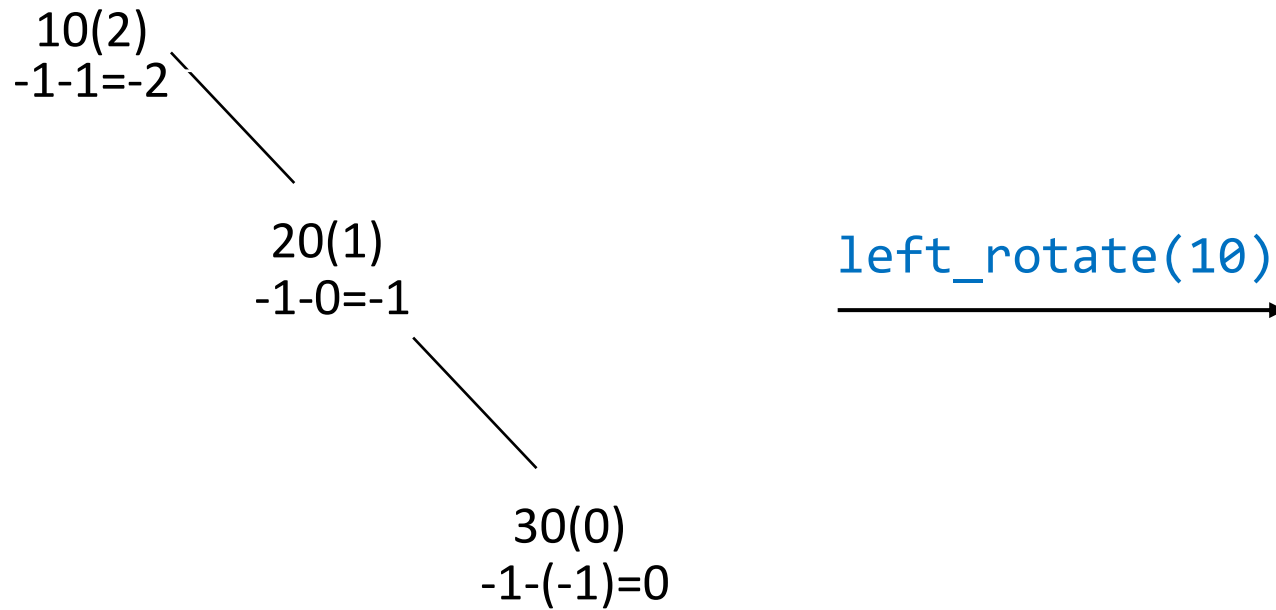
Where has newNode (30) gone?
newNode > z.Right



10 is imbalanced (z)
Z is Right-heavy

Where has newNode (30) gone?
newNode > z.Right

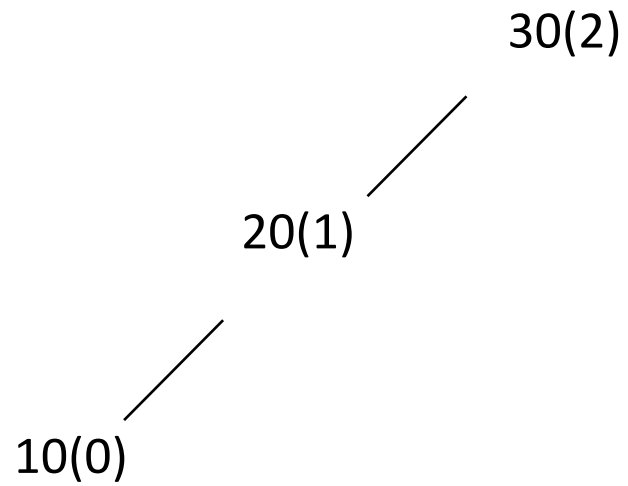
RR case!



10 is imbalanced (z)
Z is Right-heavy

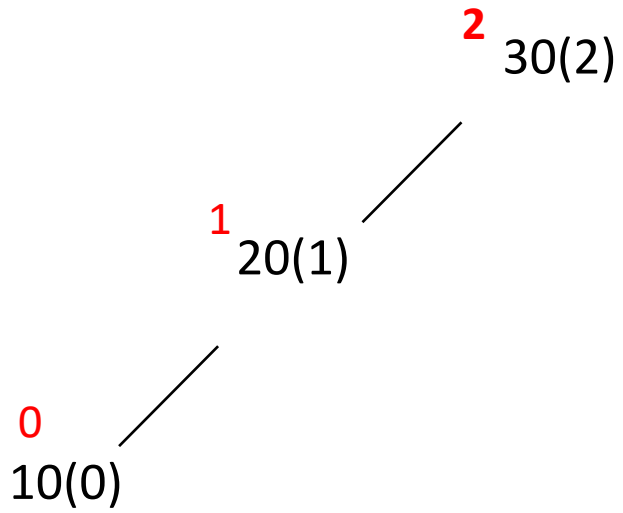
Where has newNode (30) gone?
newNode > z.Right

RR case!



30 is imbalanced (z)
Z is left-heavy

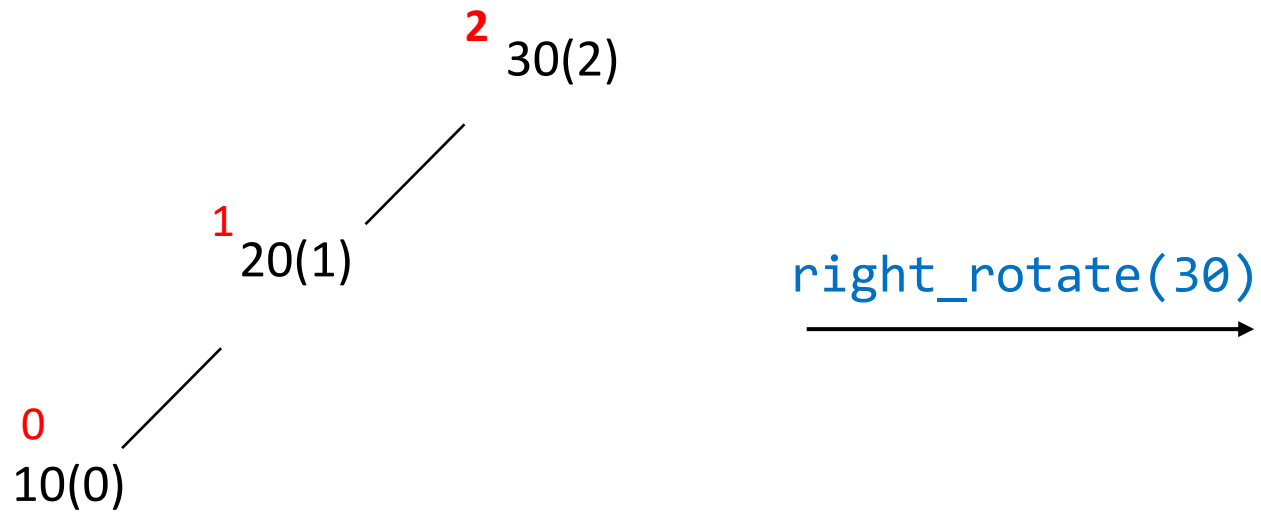
Where has newNode(10) gone?
newNode < z.left



30 is imbalanced (z)
Z is left-heavy

Where has newNode(10) gone?
newNode < z.left

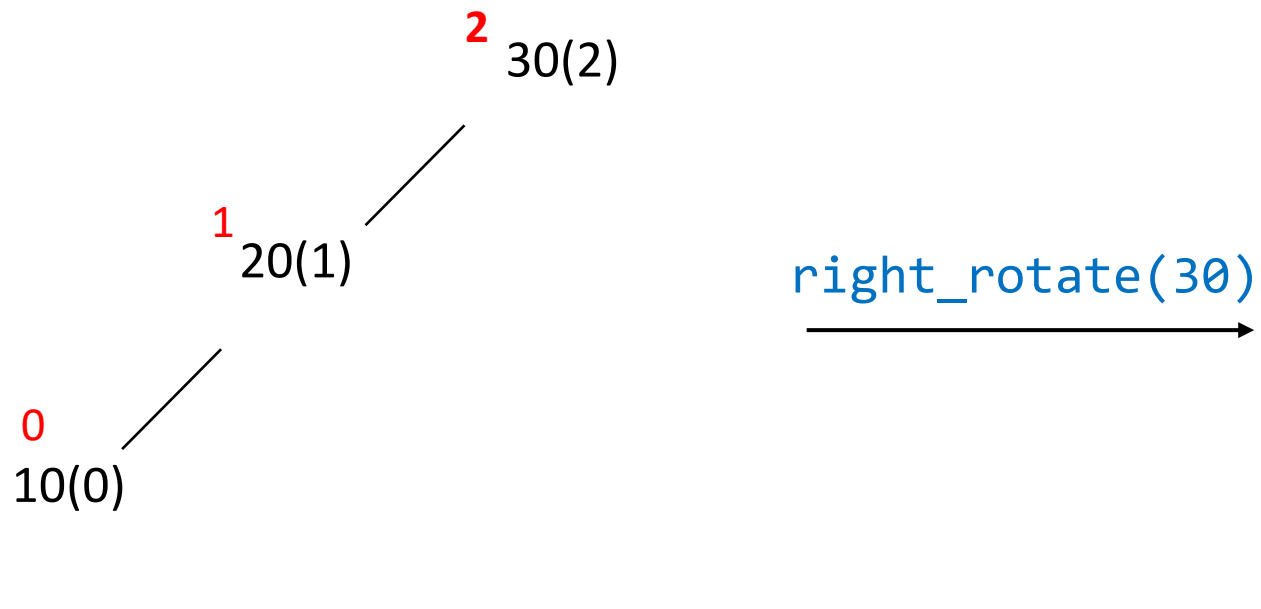
LL case!



30 is imbalanced (z)
Z is left-heavy

Where has `newNode(10)` gone?
`newNode < z.left`

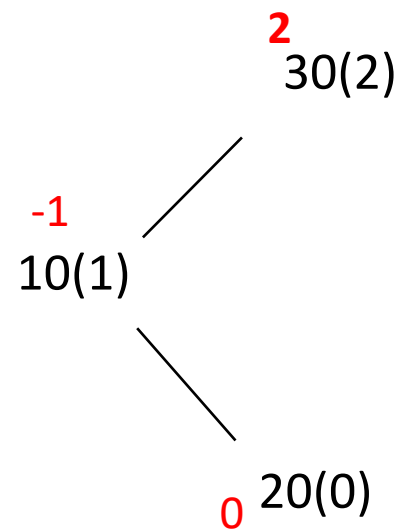
LL case!



30 is imbalanced (z)
Z is left-heavy

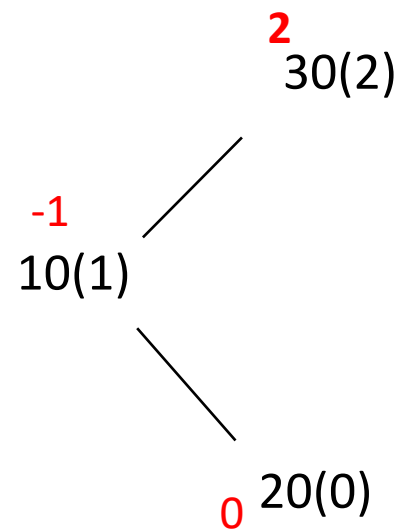
Where has newNode(10) gone?
newNode < z.left

LL case!



30 is imbalanced (z)
Z is left-heavy

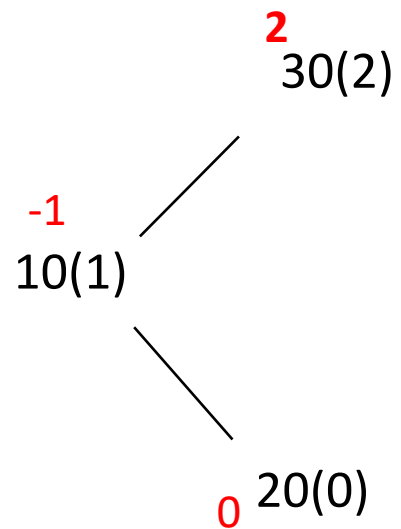
Where has newNode(20) gone?
newNode > z.left



30 is imbalanced (z)
Z is left-heavy

Where has newNode(20) gone?
newNode > z.left

LR case!



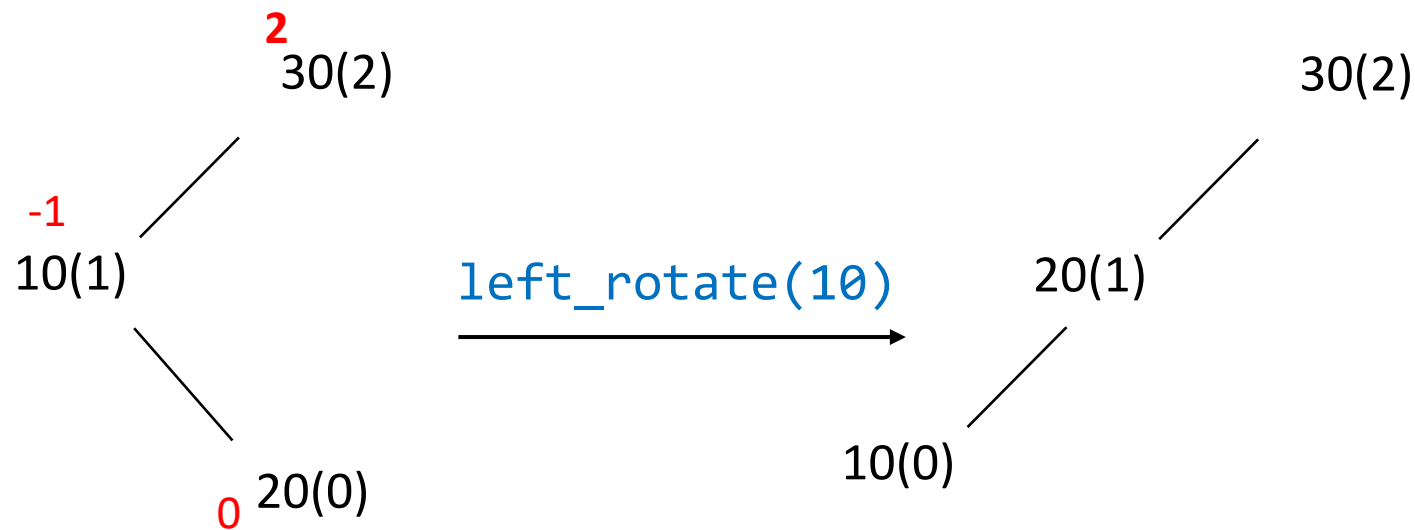
30 is imbalanced (z)
Z is left-heavy

Where has newNode(20) gone?
newNode > z.left

LR case!

Left_rotate(z.left)

right_rotate(z)



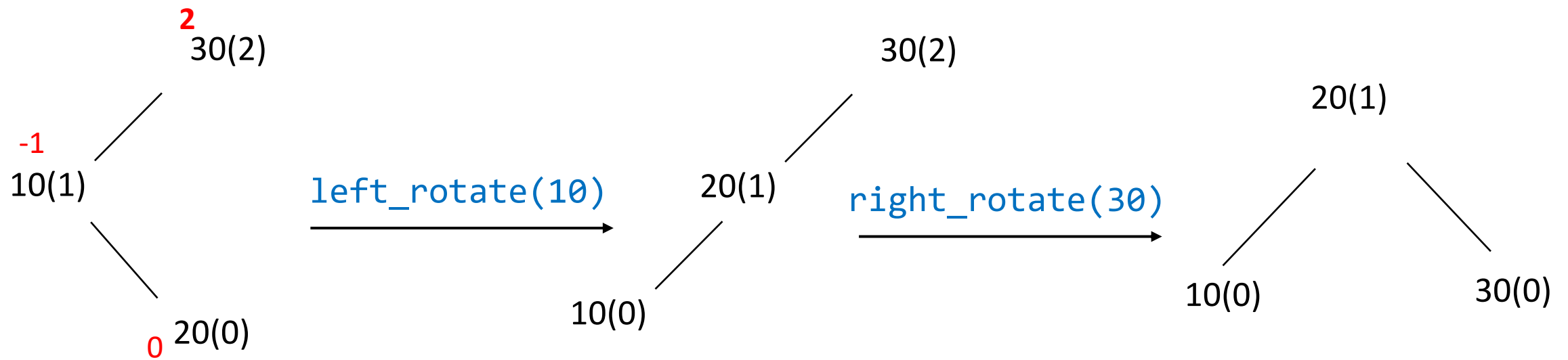
30 is imbalanced (z)
Z is left-heavy

Where has newNode(20) gone?
newNode > z.left

LR case!

Left_rotate(z.left)

right_rotate(z)



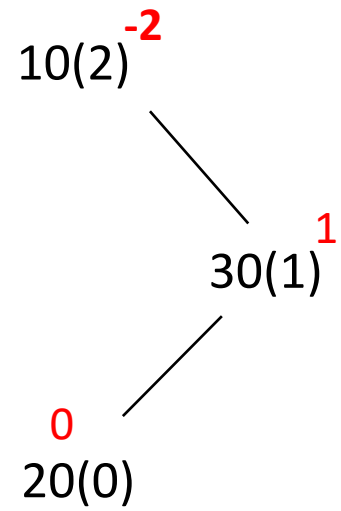
30 is imbalanced (z)
Z is left-heavy

Where has newNode gone?
newNode > z.left

LR case!

`Left_rotate(z.left)`

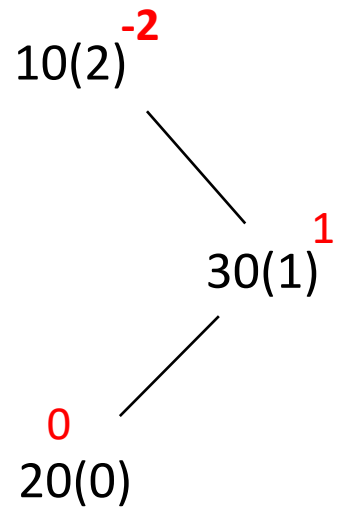
`right_rotate(z)`



10 is imbalanced (z)
Z is right-heavy

Where has newNode(20) gone?
newNode < z.right

RL case!



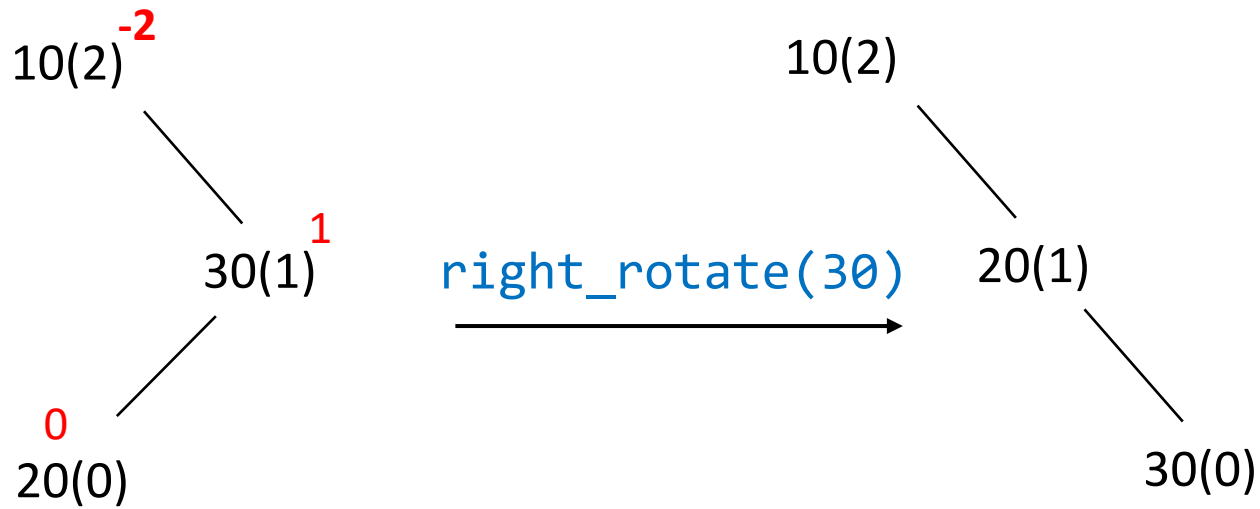
10 is imbalanced (z)
Z is right-heavy

Where has newNode(20) gone?
newNode < z.right

RL case!

right_rotate(z.right)

left_rotate(z)



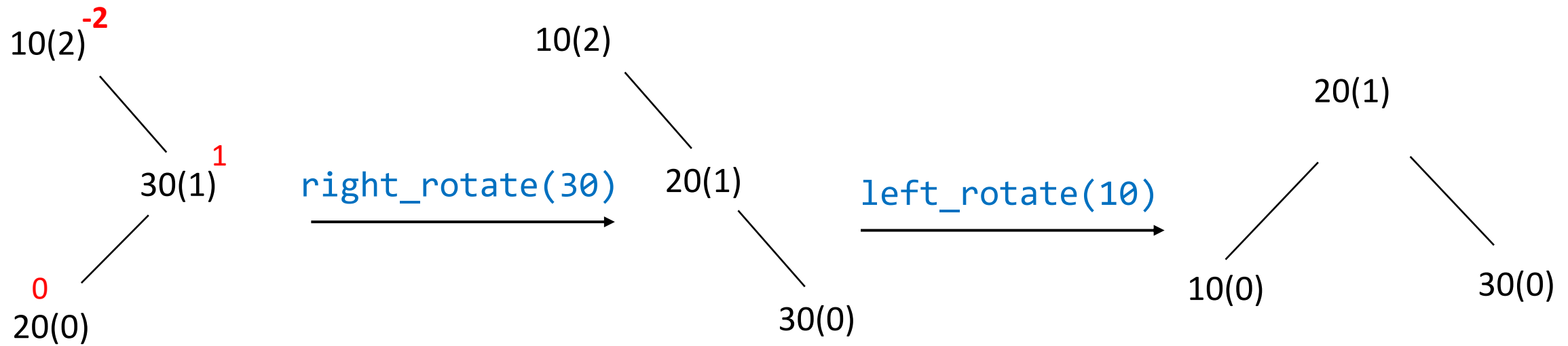
10 is imbalanced (z)
Z is right-heavy

Where has newNode(20) gone?
newNode < z.right

RL case!

right_rotate(z.right)

left_rotate(z)



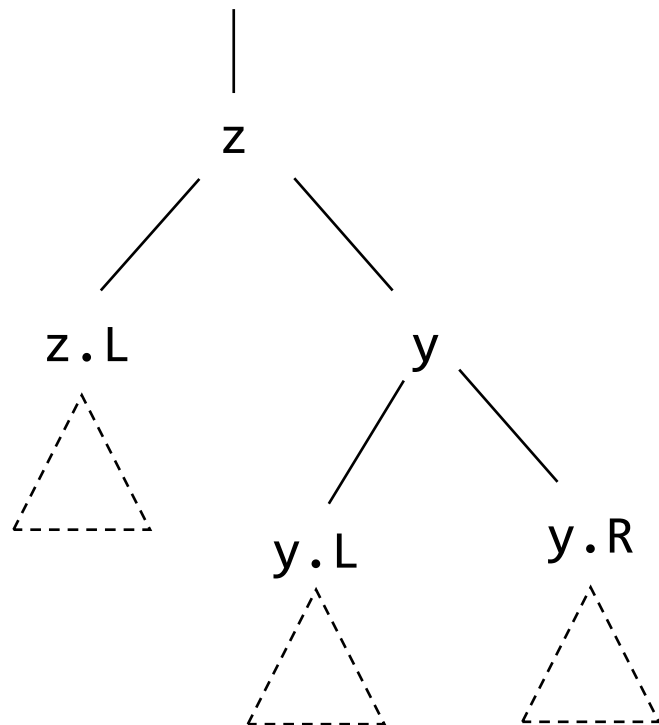
10 is imbalanced (z)
Z is right-heavy

Where has newNode(20) gone?
newNode < z.right

RL case!

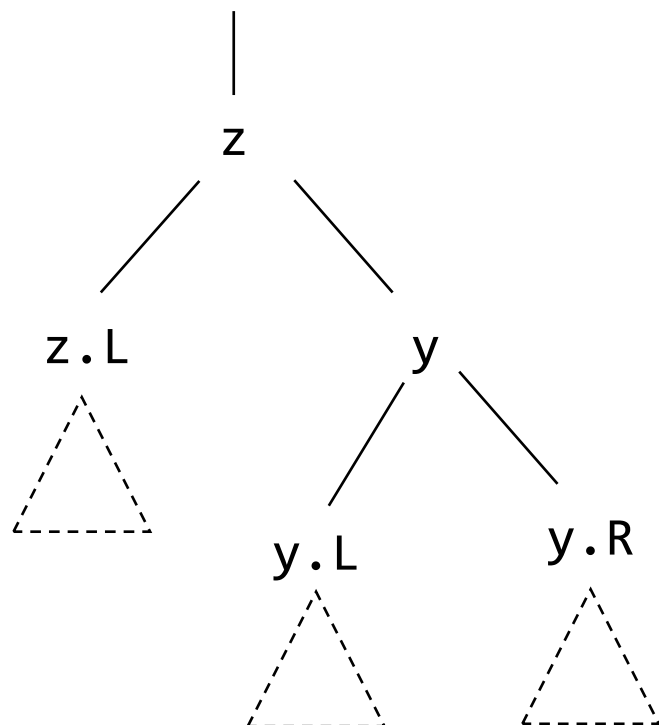
right_rotate(z.right)

left_rotate(z)

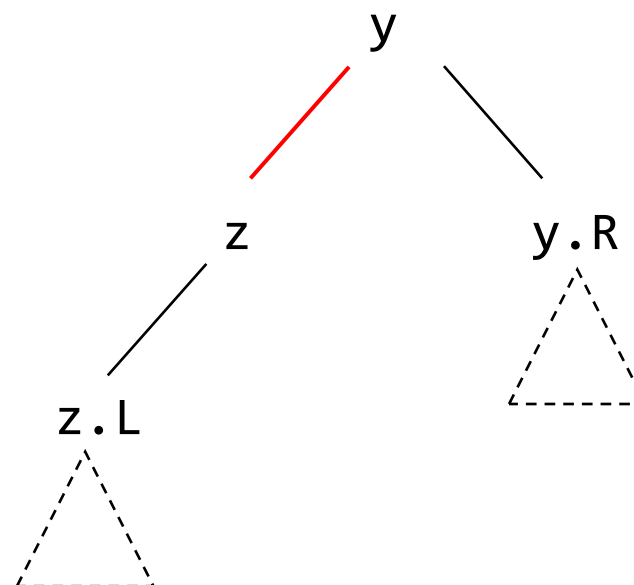


left_rotate(z)

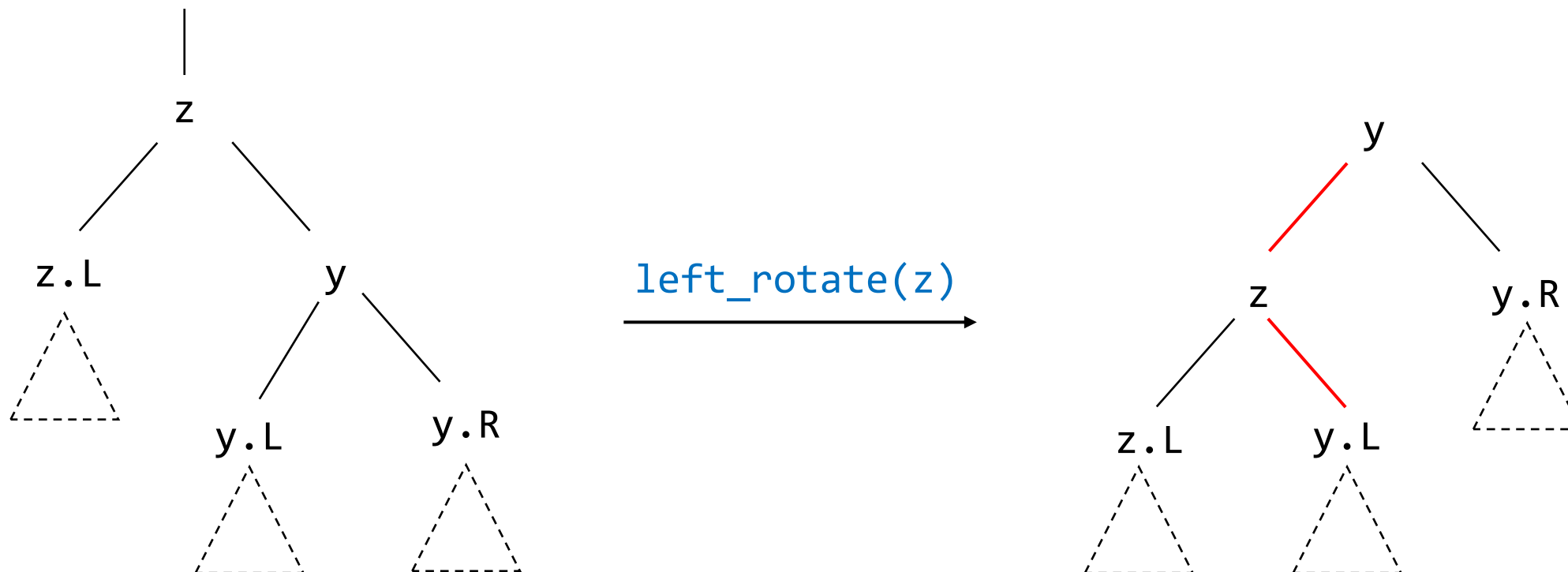




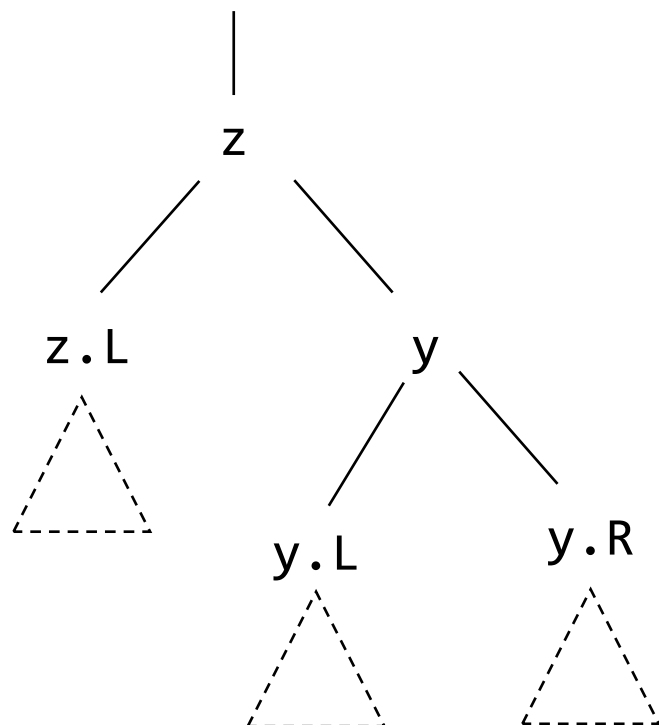
`left_rotate(z)`



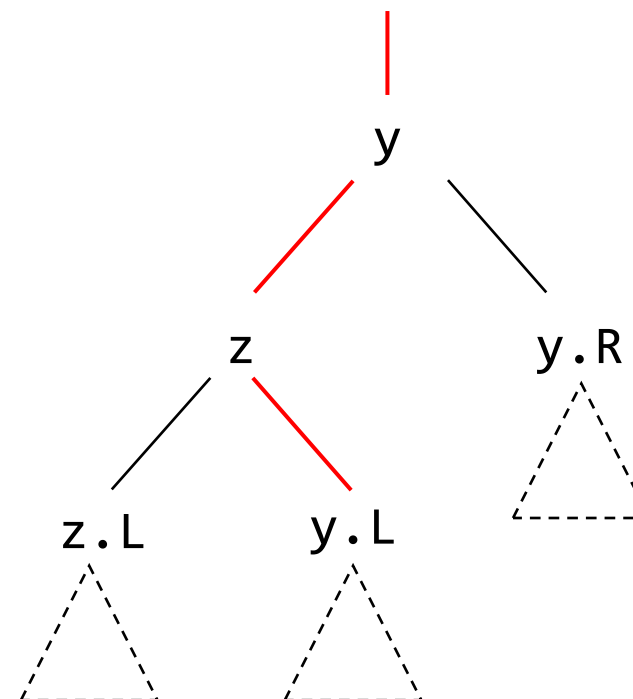
`z` goes to left of `y`



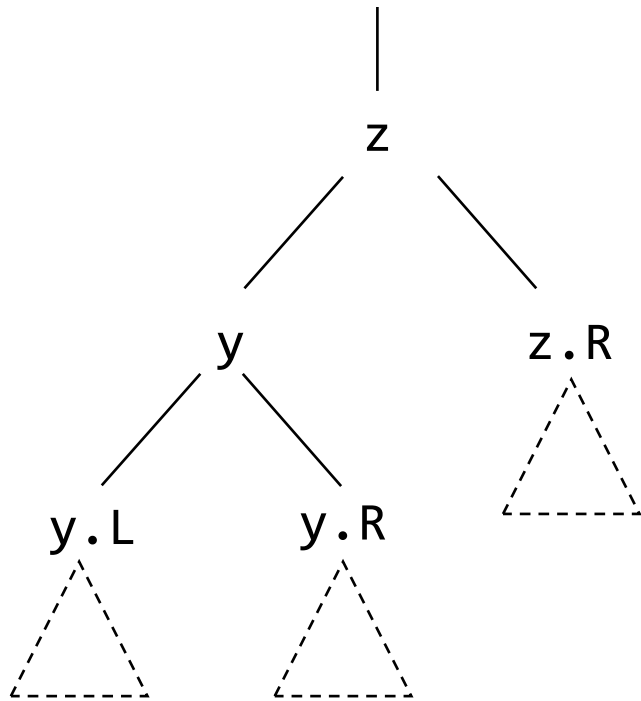
`y.L` is orphan!
It gets connected as `z`'s right



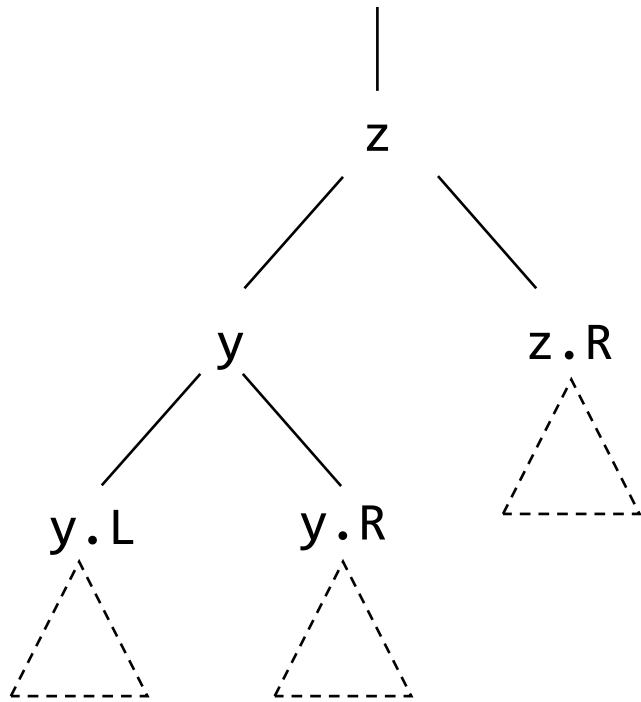
`left_rotate(z)`



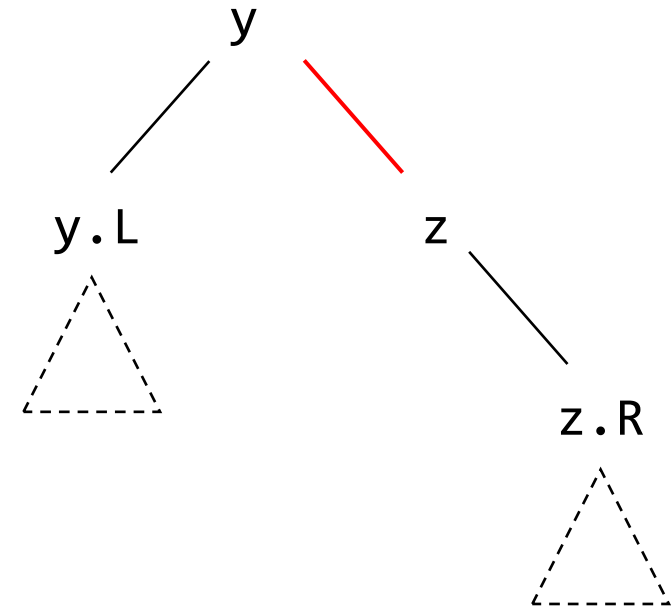
Connect y with $z.p$



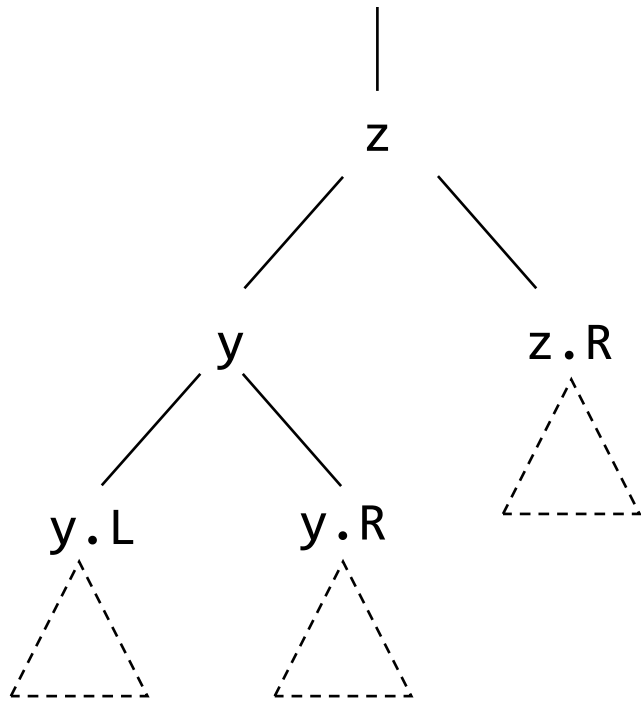
`right_rotate(z)`



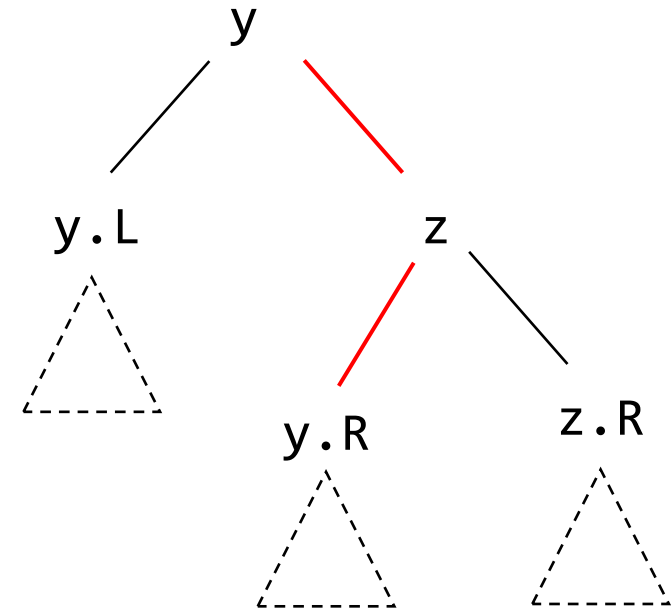
`right_rotate(z)`



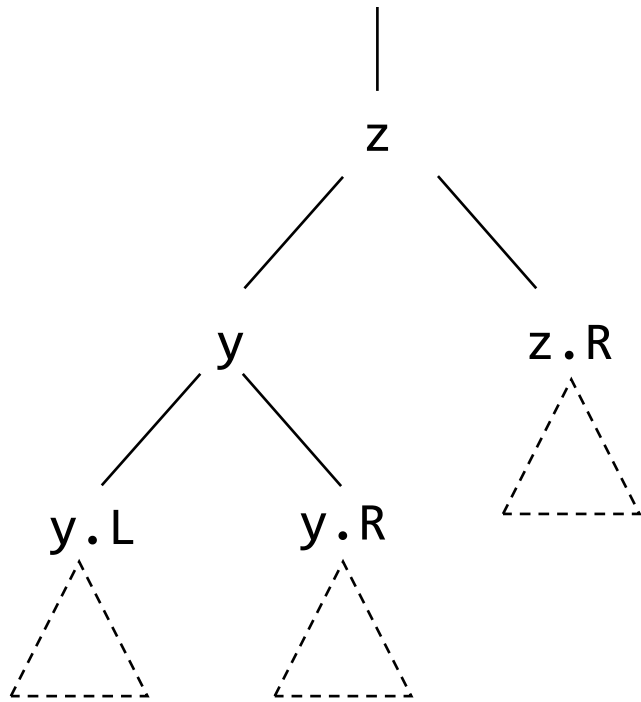
`z goes to right of y`



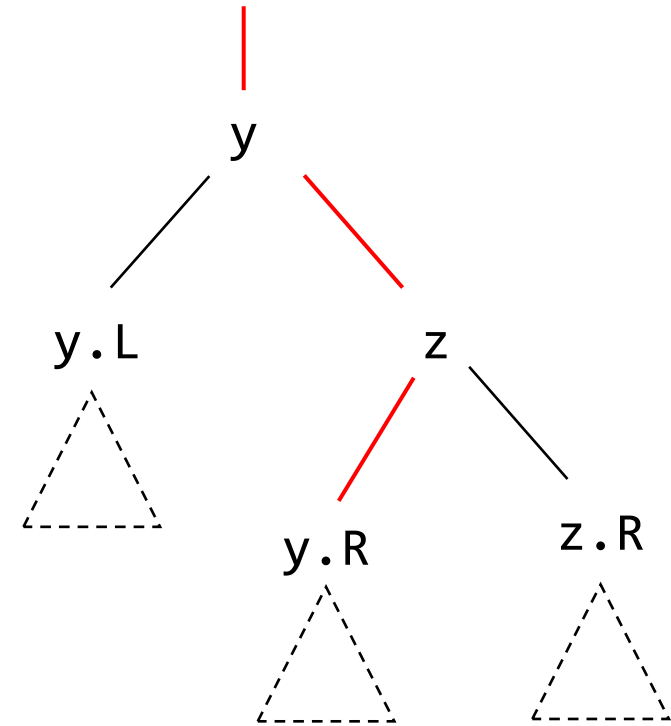
`right_rotate(z)`



`y.R` is orphan!
It gets connected as z 's left

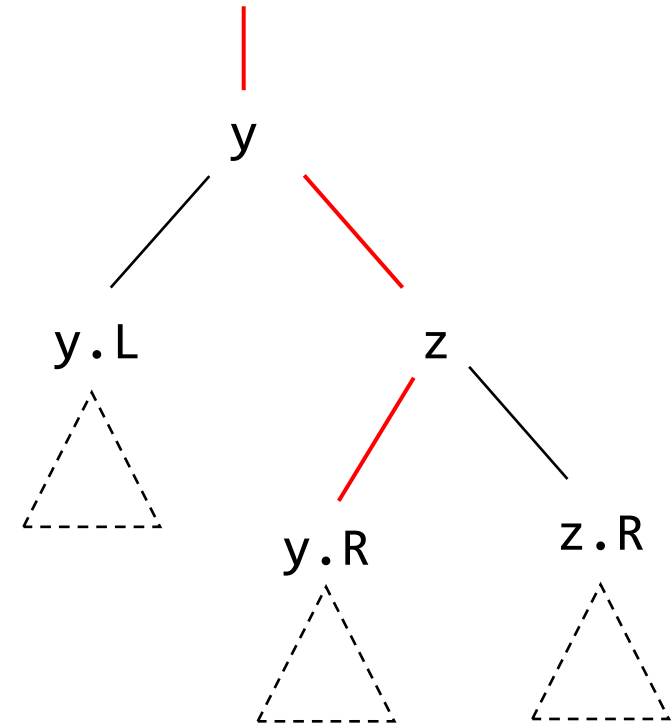


`right_rotate(z)`



Connect **y** with **z.p**

```
Void right_rotate(Node *z)
    Node *y = z.left
    Node *yR = y.right
```



```

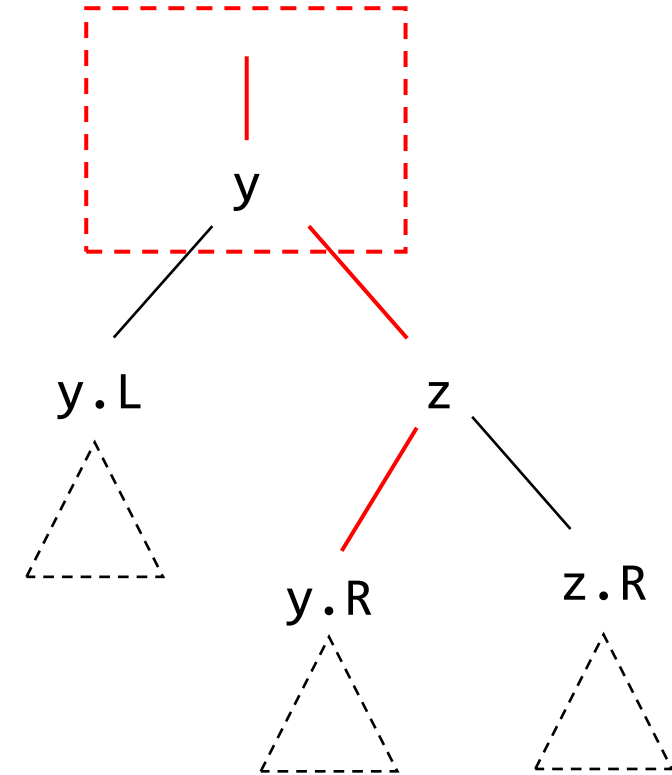
Void right_rotate(Node *z)
    Node *y = z.left
    Node *yR = y.right

```

```

    if (z==root)    root = y
    y.p=z.p
    if (z== z.p.right)    z.p.right= y
    else                    z.p.left = y

```



```

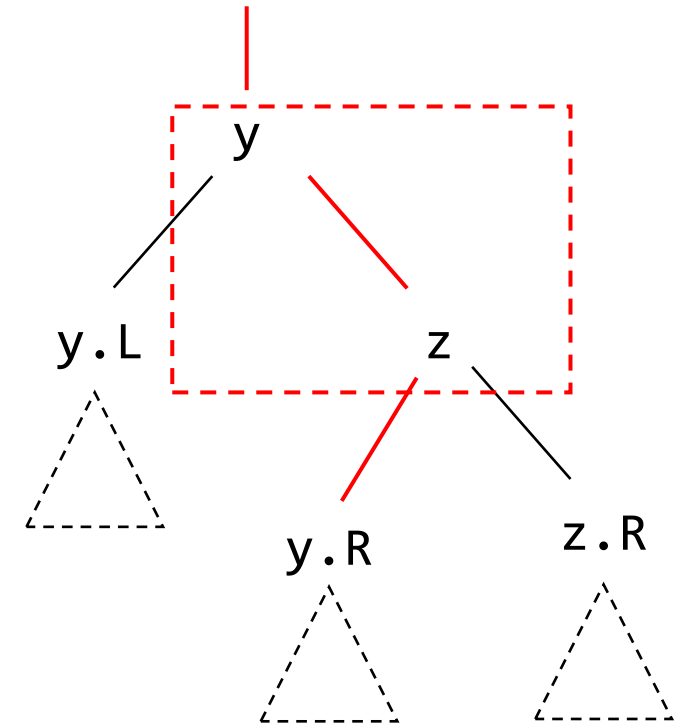
Void right_rotate(Node *z)
    Node *y = z.left
    Node *yR = y.right

    if (z==root)    root = y
    y.p=z.p

    if (z== z.p.right)    z.p.right= y
    else                    z.p.left = y

    y.right=z
    z.parent=y

```



```

Void right_rotate(Node *z)
    Node *y = z.left
    Node *yR = y.right

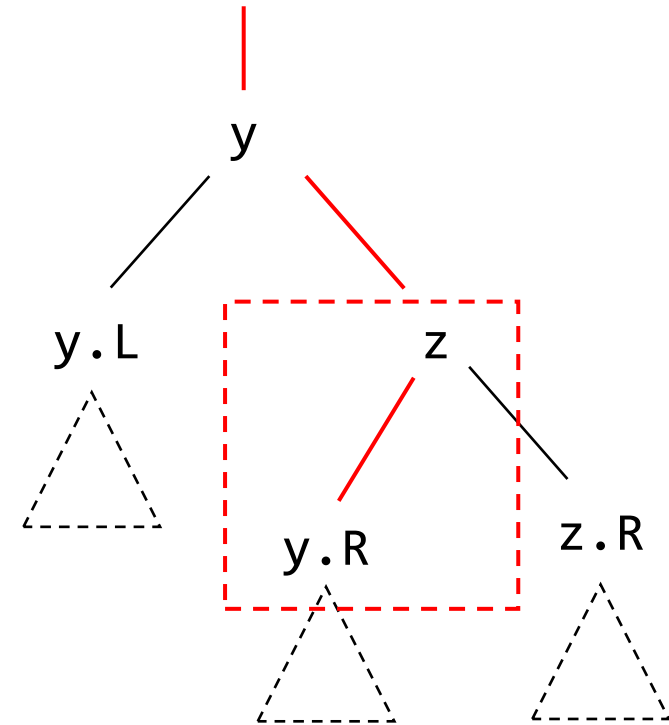
    if (z==root)    root = y
    y.p=z.p

    if (z== z.p.right)    z.p.right= y
    else                    z.p.left = y

    y.right=z
    z.parent=y

    z.left=yR
    if(yR!=Null) yR.p=z

```



```

Void right_rotate(Node *z)
    Node *y = z.left
    Node *yR = y.right

    if (z==root)    root = y
    y.p=z.p

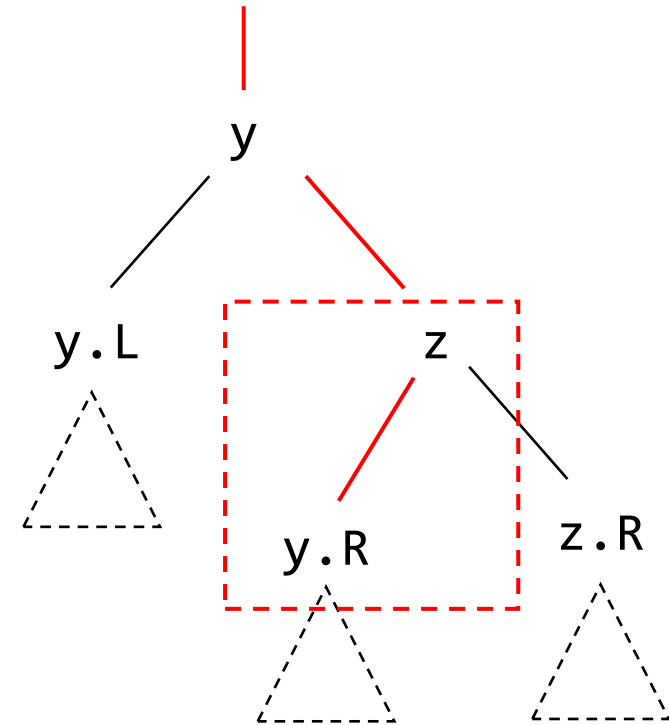
    if (z== z.p.right)    z.p.right= y
    else                    z.p.left = y

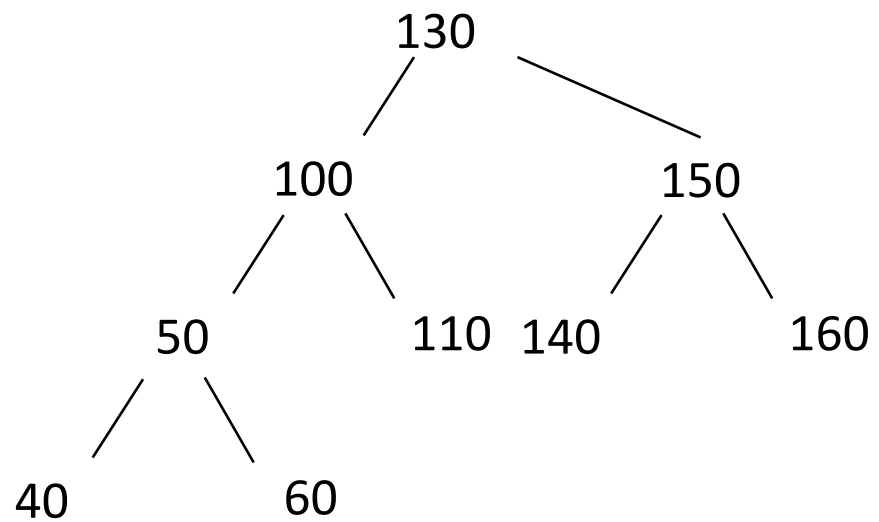
    y.right=z
    z.parent=y

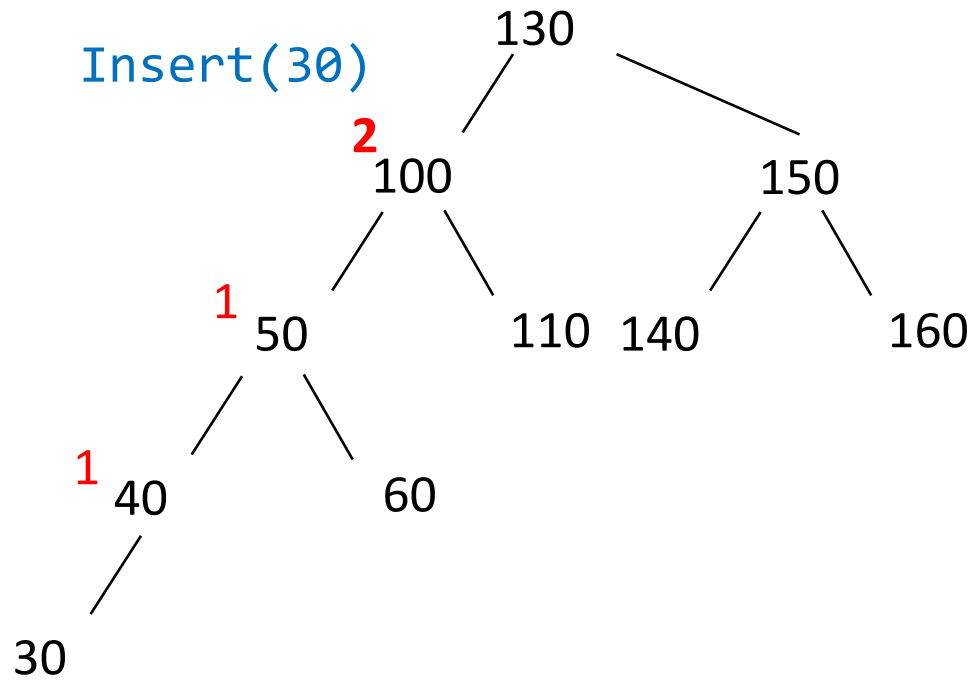
    z.left=yR
    if(yR!=Null) yR.p=z

    update_ancestor_heights(z)

```

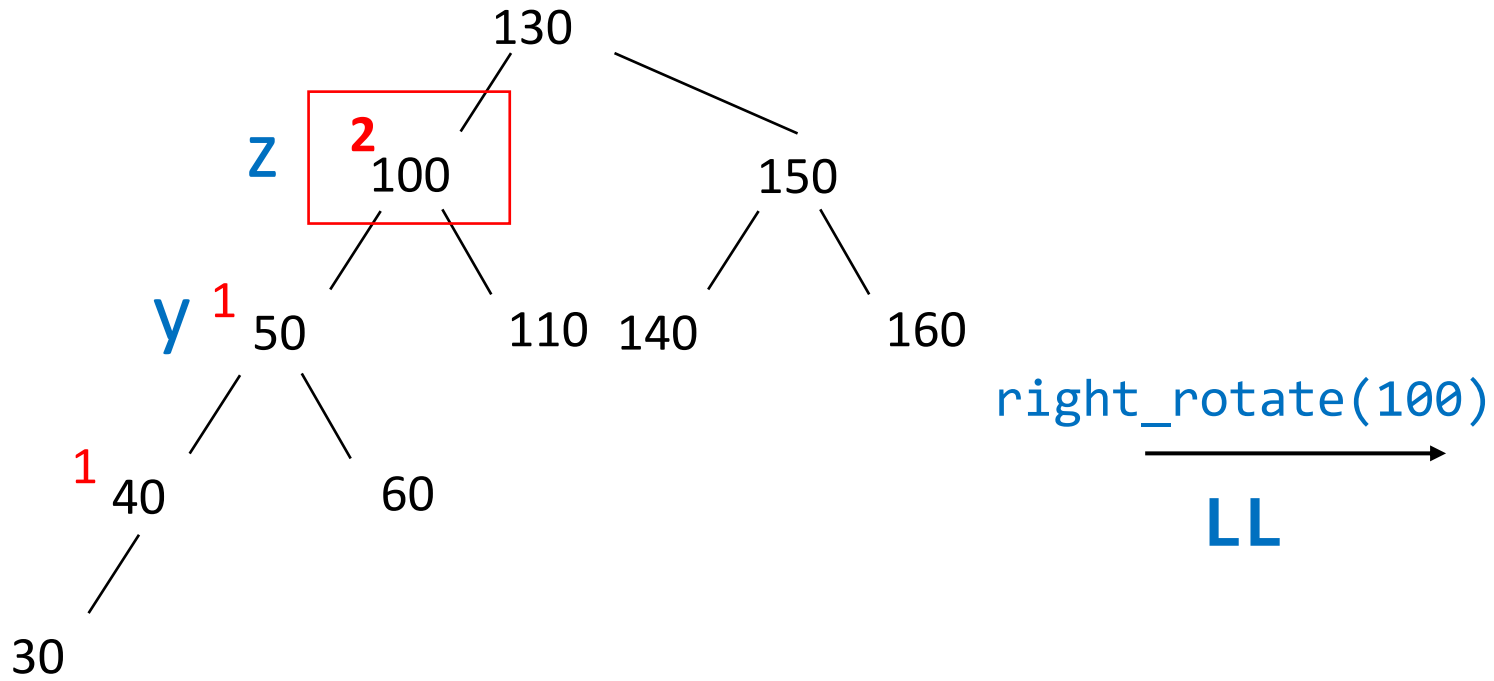






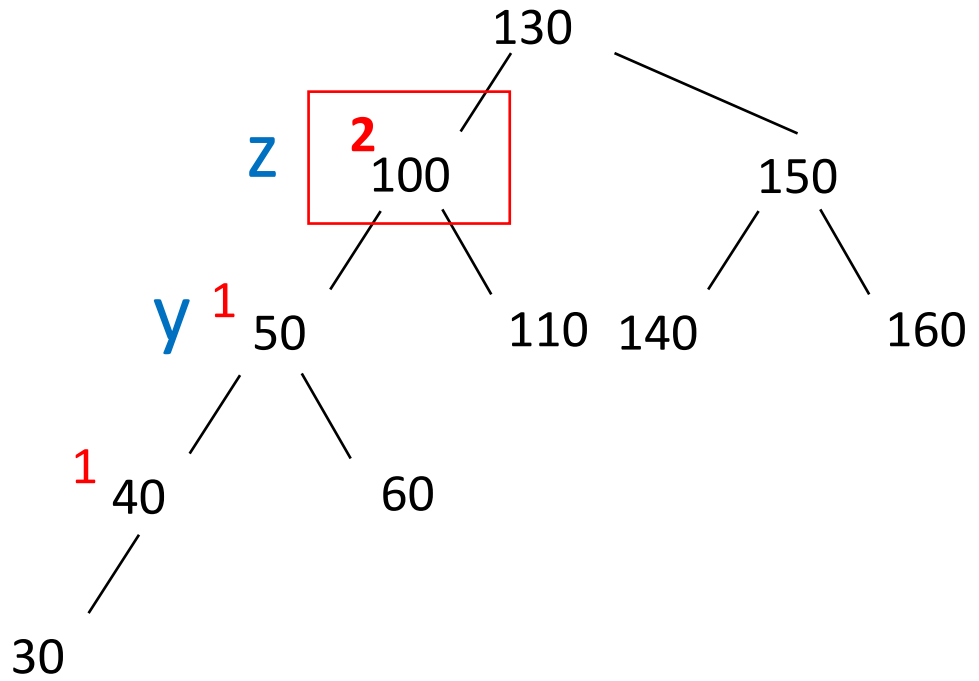
z= 100 is imbalanced, left heavy
newNode(30) < z.left

LL case



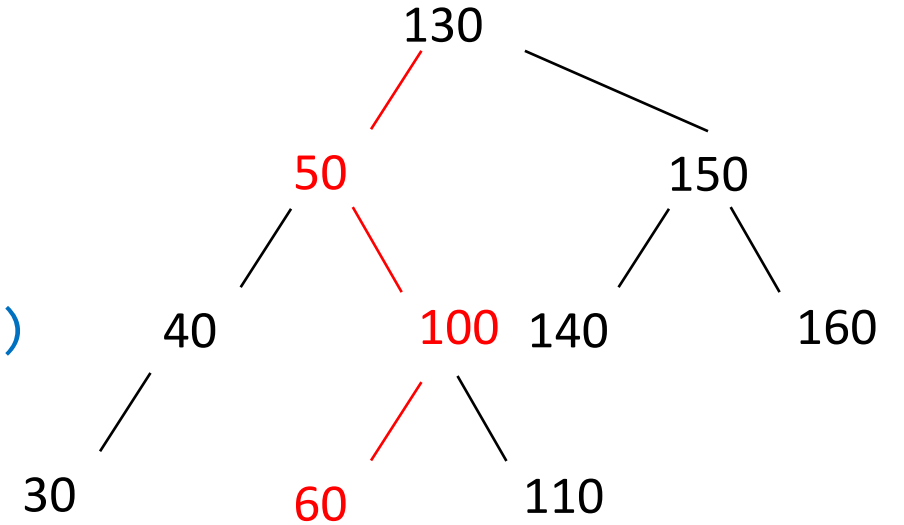
z= 100 is imbalanced, left heavy
 newNode(30) < z.left

LL case



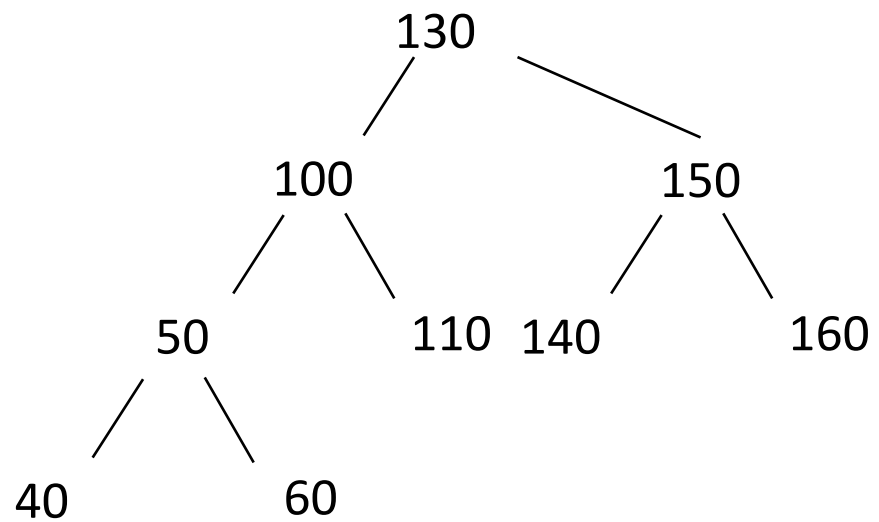
right_rotate(100)

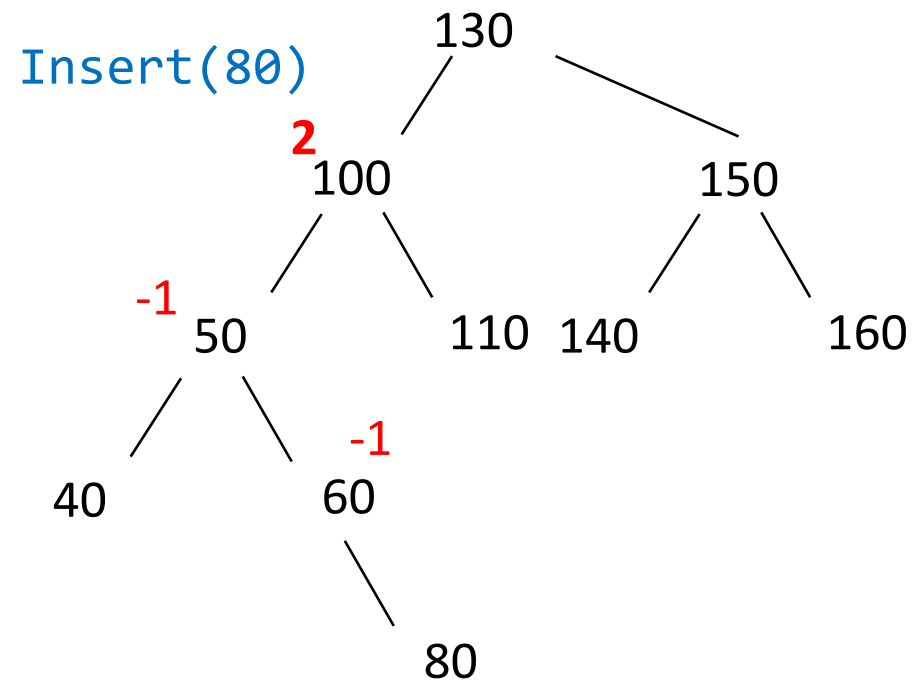
LL

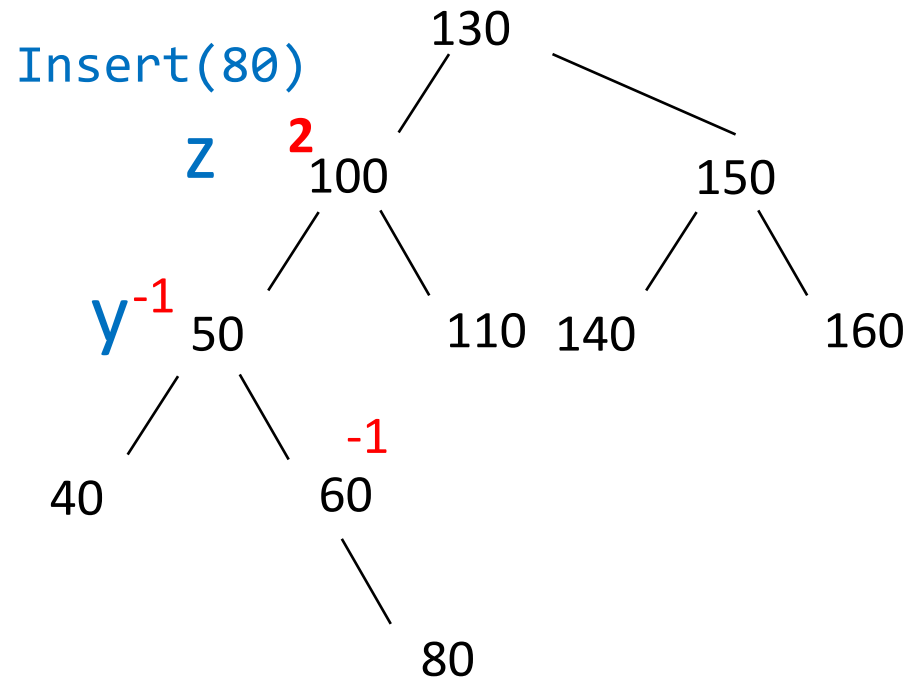


z= 100 is imbalanced, left heavy
 newNode(30) < z.left

LL case

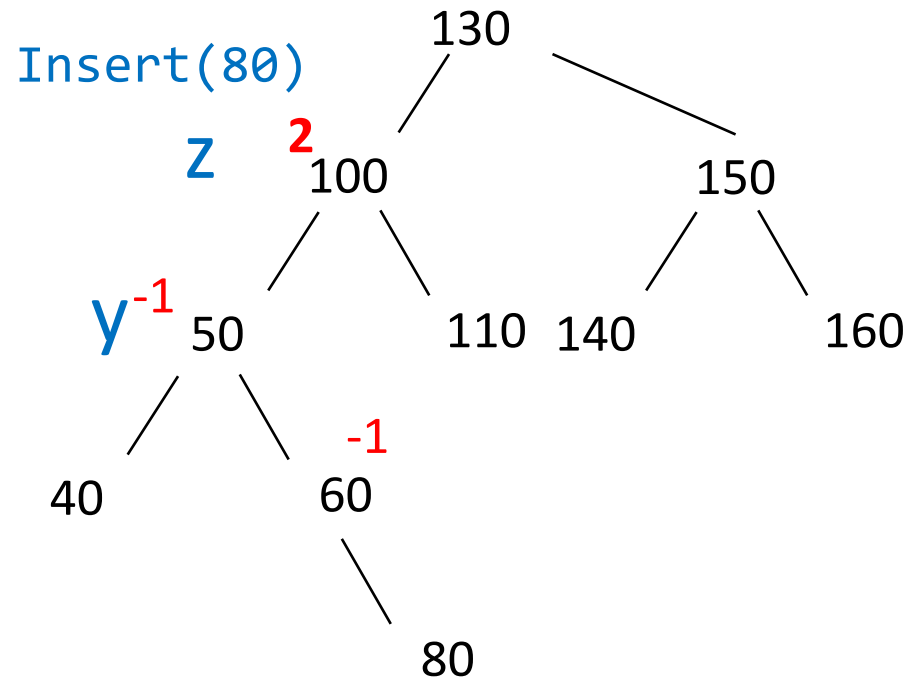






z= 100 is imbalanced, left heavy
newNode(80) > z.left

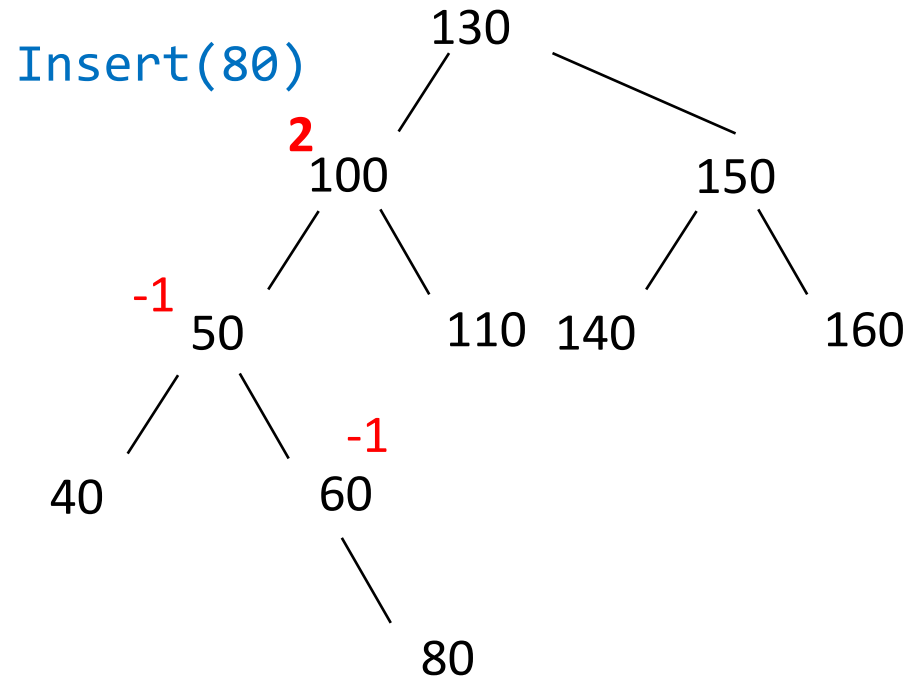
LR case



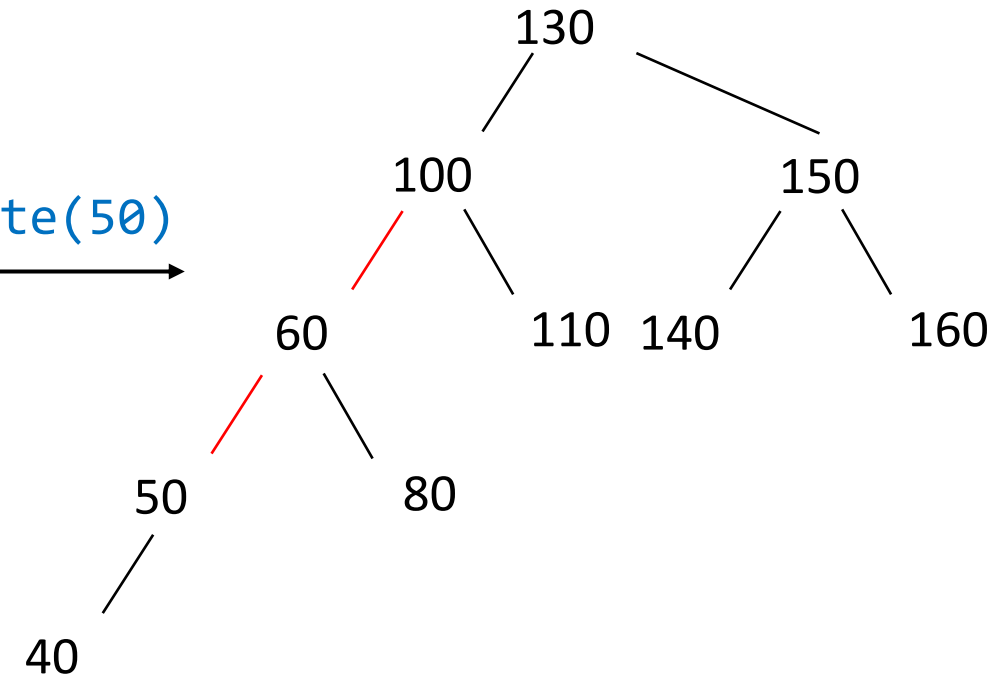
z= 100 is imbalanced, left heavy
newNode(80) > z.left

LR case

Left_rotate(y)
right_rotate(z)



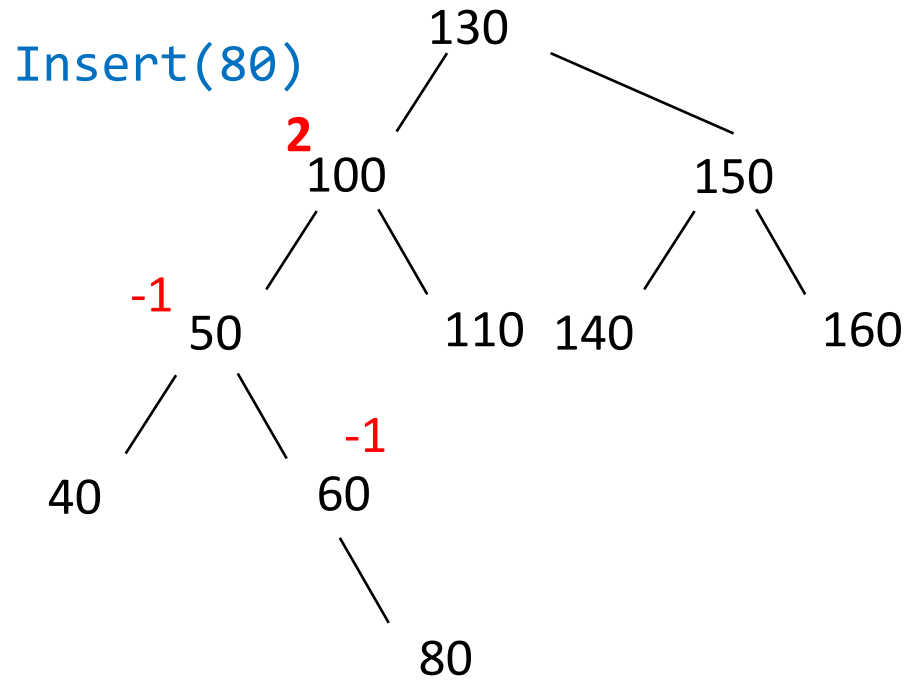
left_rotate(50)



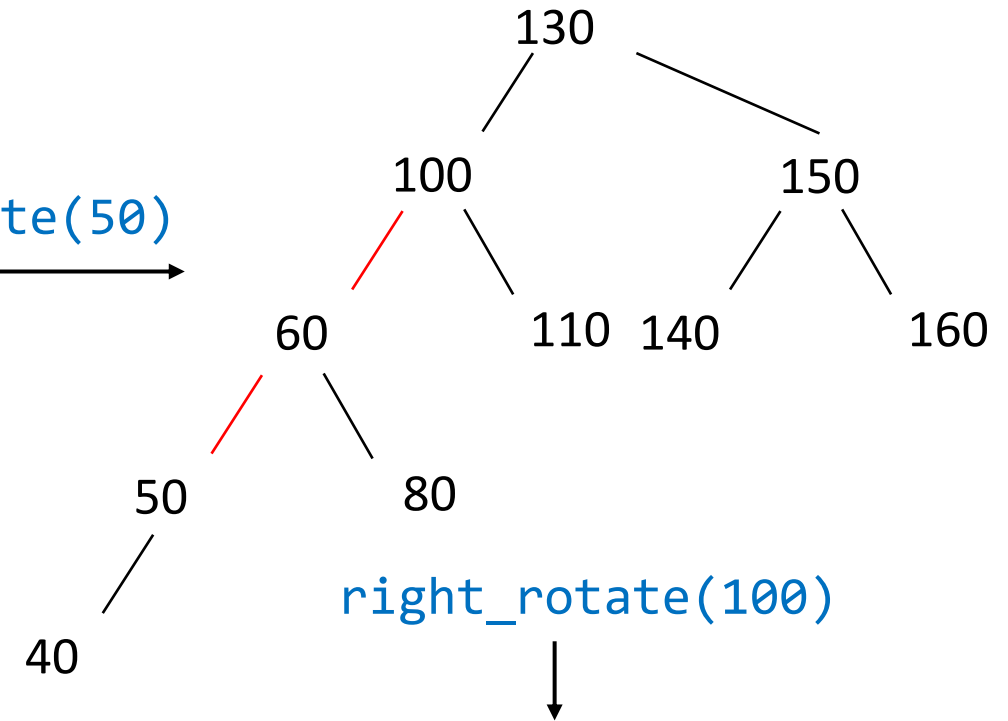
z = 100 is imbalanced, left heavy
 newNode(80) > z.left

LR case

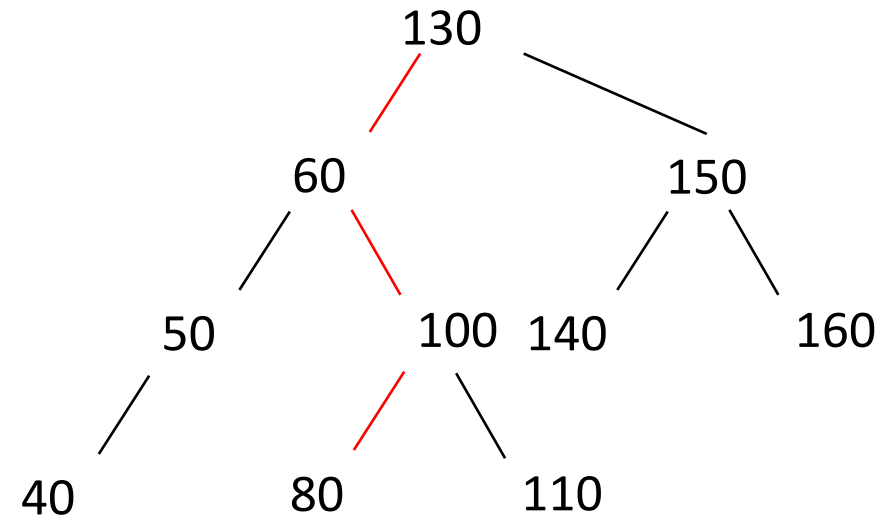
Left_rotate(y)
 right_rotate(z)



left_rotate(50)



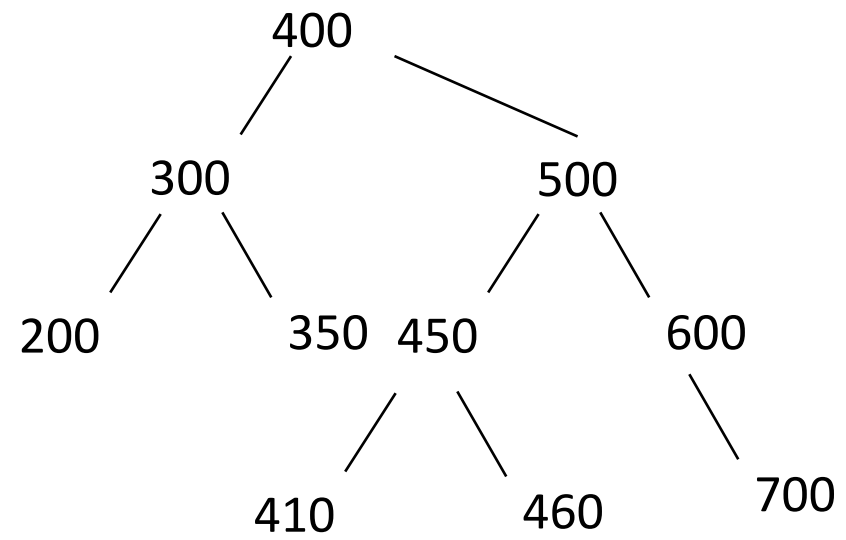
right_rotate(100)

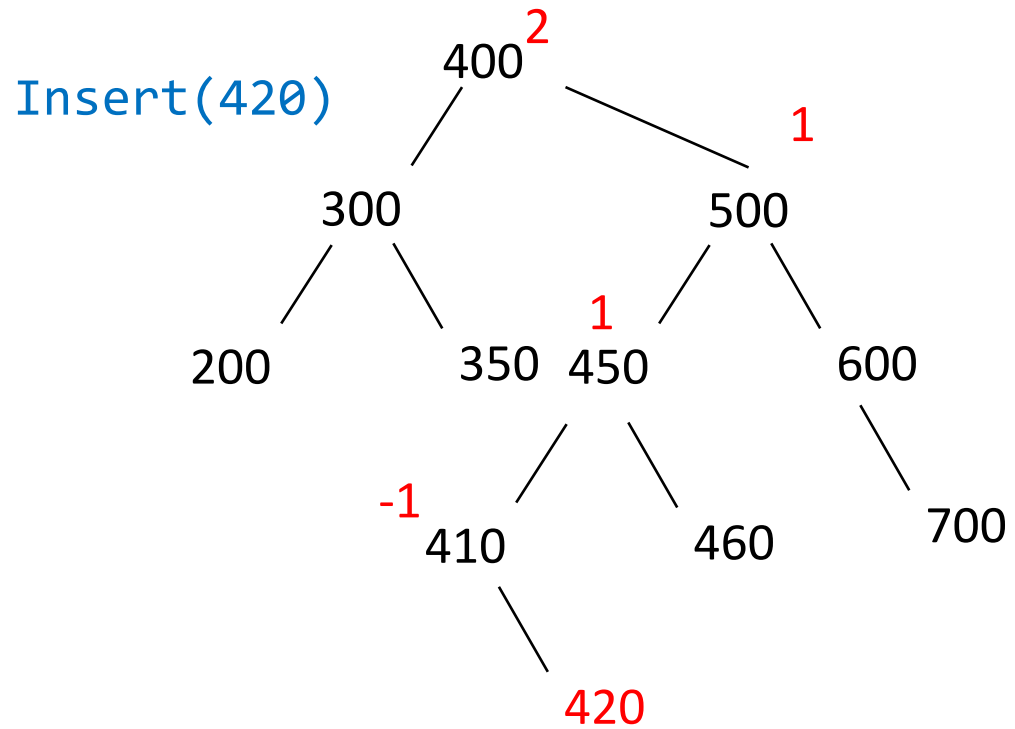


z = 100 is imbalanced, left heavy
 newNode(80) > z.left

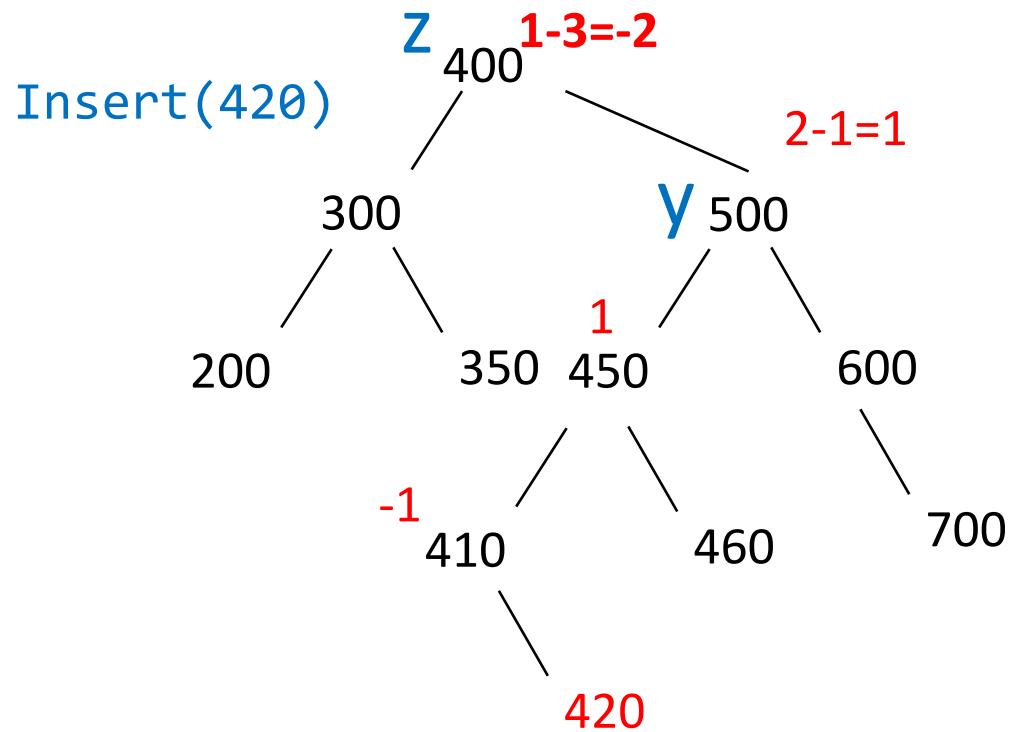
LR case

Left_rotate(y)
 right_rotate(z)



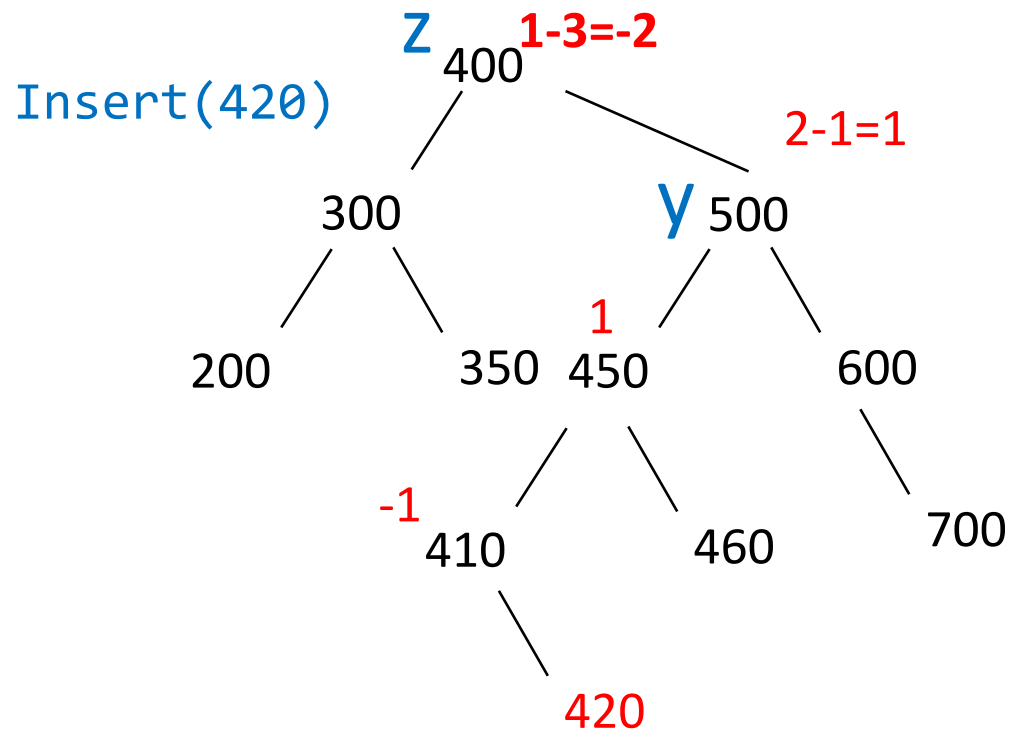


Due to the insertion in a deeper level, root got imbalanced!



Due to the insertion in a deeper level, root got imbalanced!

$z = 400$ is imbalanced, right heavy
 $\text{newNode}(420) < z.\text{right}$



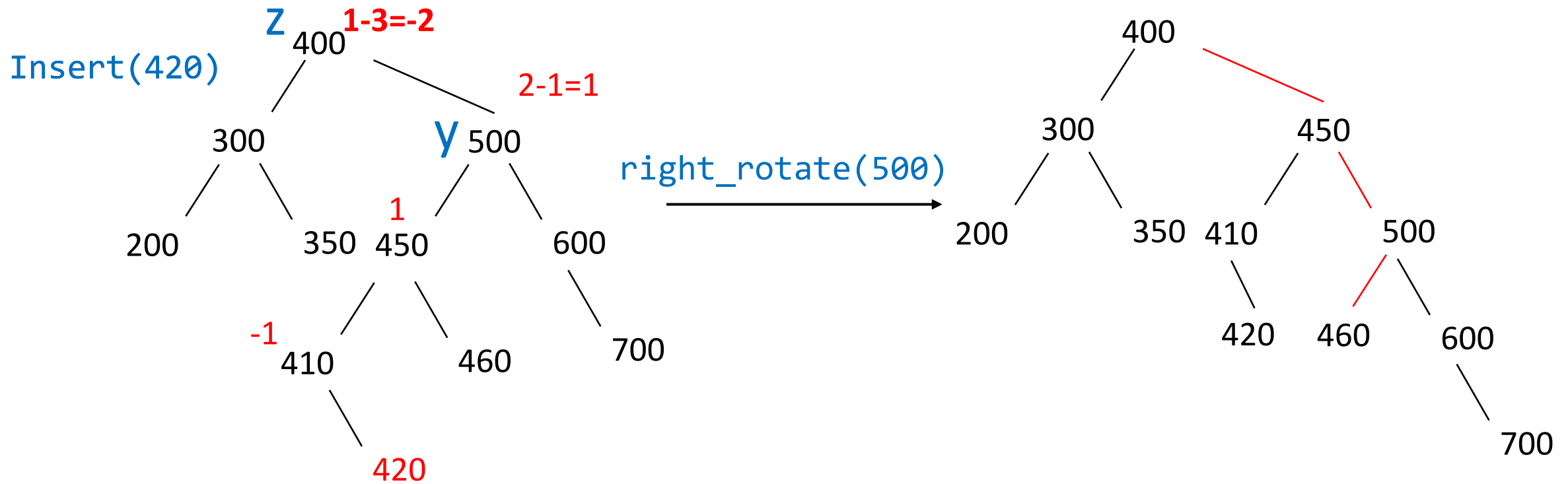
Due to the insertion in a deeper level, root got imbalanced!

z= 400 is imbalanced, right heavy
 newNode(420) < z.right

RL case

right_rotate(y)

Left_rotate(z)



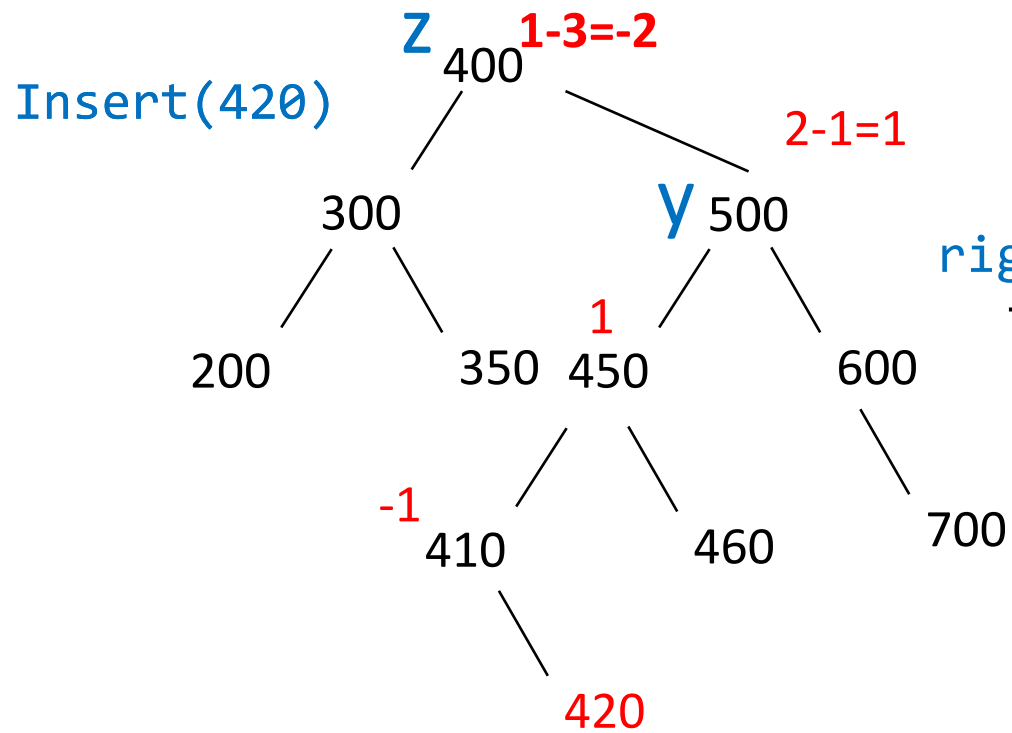
Due to the insertion in a deeper level, root got imbalanced!

z = 400 is imbalanced, right heavy
 newNode(420) < z.right

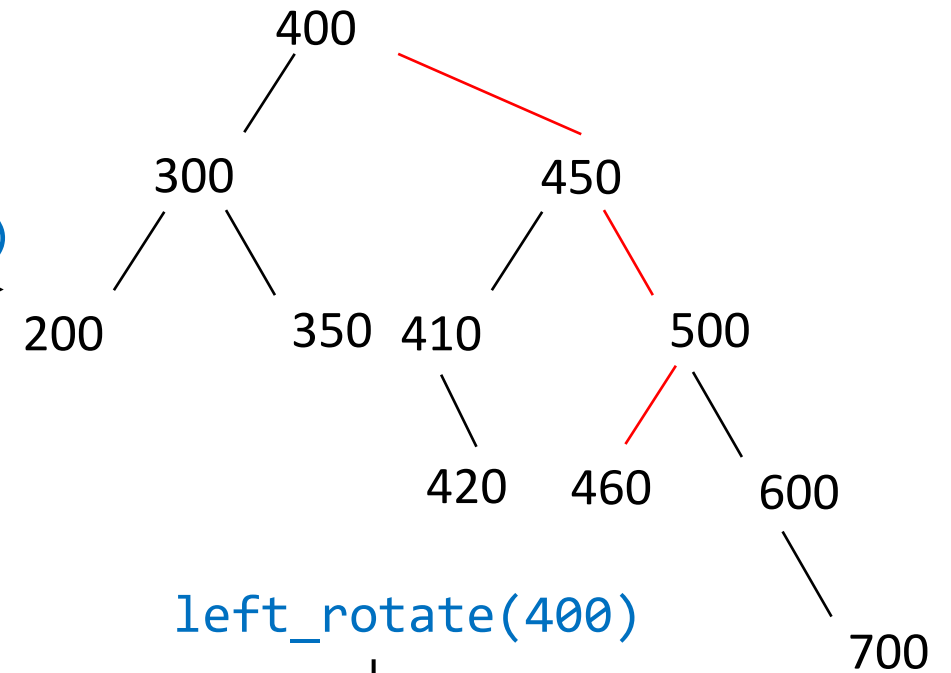
RL case

right_rotate(y)

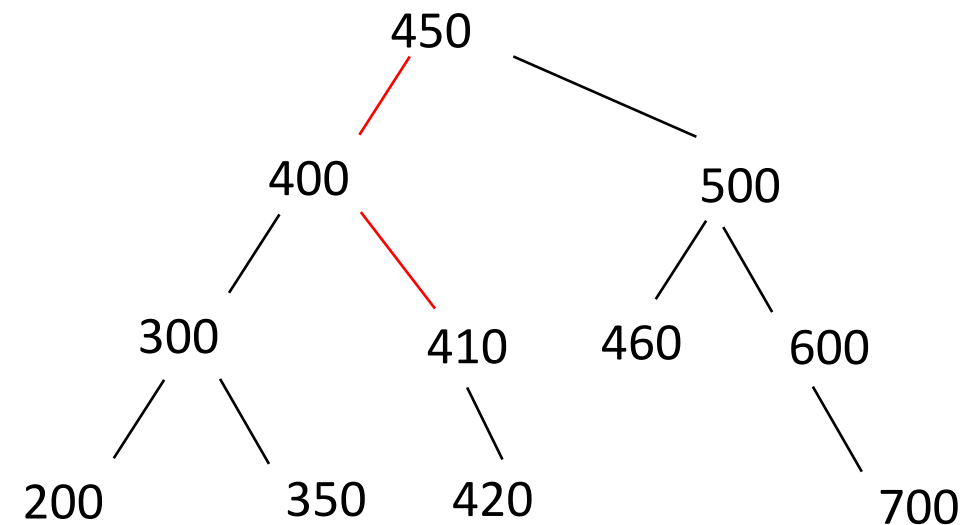
Left_rotate(z)



right_rotate(500)



left_rotate(400)



Due to the insertion in a deeper level, root got imbalanced!

$z = 400$ is imbalanced, right heavy
 $\text{newNode}(420) < z.\text{right}$

RL case

right_rotate(y)

Left_rotate(z)

Meme alert!

After Learning Tree



After Learning Tree



After Learning AVL tree



```

Void right_rotate(Node *z)
    Node *y = z.left
    Node *yR = y.right

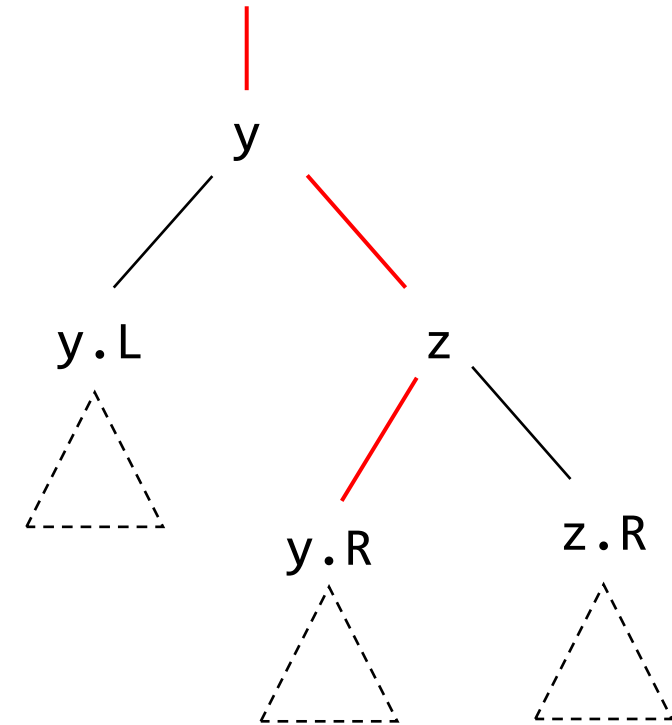
    if (z==root)    root = y
    y.p=z.p
    if (z== z.p.right)    z.p.right= y
    else                    z.p.left = y

    y.right=z
    z.parent=y

    z.left=yR
    if(yR!=Null) yR.p=z

    update_ancestor_heights(z)

```



```
update_ancestor_heights(Node *z)
    temp = z
    while temp!=Null
        update_height(temp)
        temp=temp.parent
```

```
update_ancestor_heights(Node *z)
    temp = z
    while temp!=Null
        update_height(temp)
        temp=temp.parent
```

```
Update_height(Node *x)
    x.height = 1+max( height(x.left), height(x.right) )
```

```

void update_ancestor_heights(Node *z)
    temp = z
    while temp!=Null
        update_height(temp)
        temp=temp.parent

void update_height(Node *x)
    x.height = 1+max( height(x.left), height(x.right) )

int height(Node *x)
    if x==Null return -1
    else      return x.height

```

```

void update_ancestor_heights(Node *z)
    temp = z
    while temp!=Null
        update_height(temp)
        temp=temp.parent

void update_height(Node *x)
    x.height = 1+max( height(x.left), height(x.right) )

int height(Node *x)
    if x==Null return -1
    else      return x.height

int balance_factor(Node *x)
    return height(x.left) - height(x.right)

```

```
balance_node(Node *z, int b_fact)
    if b_fact>1
```

```
balance_node(Node *z, int b_fact)
    if b_fact>1
        if newNode.data < z.left.data    // LL
            right_rotate(z)
```

```
balance_node(Node *z, int b_fact)
    if b_fact>1
        if newNode.data < z.left.data    // LL
            right_rotate(z)
        if newNode.data > z.left.data    // LR
            left_rotate(z.left)
            right_rotate(z)
```

```
balance_node(Node *z, int b_fact)
    if b_fact>1
        if newNode.data < z.left.data    // LL
            right_rotate(z)
        if newNode.data > z.left.data    // LR
            left_rotate(z.left)
            right_rotate(z)

    if b_fact<-1
```

```

balance_node(Node *z, int b_fact)
    if b_fact>1
        if newNode.data < z.left.data    // LL
            right_rotate(z)
        if newNode.data > z.left.data    // LR
            left_rotate(z.left)
            right_rotate(z)

    if b_fact<-1
        if newNode.data > z.right.data    // RR
            left_rotate(z)

```

```

balance_node(Node *z, int b_fact)
    if b_fact>1
        if newNode.data < z.left.data    // LL
            right_rotate(z)
        if newNode.data > z.left.data    // LR
            left_rotate(z.left)
            right_rotate(z)

    if b_fact<-1
        if newNode.data > z.right.data    // RR
            left_rotate(z)
        if newNode.data < z.right.data    // RL
            right_rotate(z.right)
            left_rotate(z)

```

`inset (int key)`

same as BST insertion

`inset (int key)`

same as BST insertion

`update_ancestor_heights(newNode)`

```
inset (int key)
```

```
    same as BST insertion
```

```
    update_ancestor_heights(newNode)
```

```
    Node* temp = newNode
```

```
    while temp!=Null
```

```
        int b_fact = balance_factor(temp)
```

`inset (int key)`

same as BST insertion

`update_ancestor_heights(newNode)`

`Node* temp = newNode`

`while temp!=Null`

`int b_fact = balance_factor(temp)`

`if b_fact<-1 || b_fact>1`

`balance_node(temp)`

`break`

`else temp=temp.parent`

References

1. GeeksforGeeks. (2024, August 8). AVL Tree Data Structure. GeeksforGeeks. <https://www.geeksforgeeks.org/introduction-to-avl-tree/>

Thank you